backend/app/main.py — Titik masuk aplikasi FastAPI (50 baris)

```
import logging
from contextlib import asynccontextmanager
from pathlib import Path

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import RedirectResponse
from fastapi.staticfiles import StaticFiles

from .config import get_settings
from .db import init_db
from .routers import admin, public

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(name)s: %(message)s")
STATIC_DIR = Path(__file__).parent / "static"


@asynccontextmanager
async def lifespan(_: FastAPI):
    init_db()
    yield


app = FastAPI(
    title="YSII — Sistem Promosi Digital Terintegrasi SEO & Machine Learning",
    description=(
        "API untuk tracking perilaku pengunjung, pendaftaran lead, prediksi konversi, "
        "rekomendasi konten, dan dashboard manajemen lead."
    ),
    version="1.0.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=get_settings().origins,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)

app.include_router(public.router)
app.include_router(admin.auth_router)
app.include_router(admin.router)
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")
app.mount("/dashboard", StaticFiles(directory=STATIC_DIR / "dashboard", html=True), name="dashboard")


@app.get("/", include_in_schema=False)
def root():
    return RedirectResponse("/dashboard/")
```

backend/app/config.py — Konfigurasi aplikasi (46 baris)

```
from functools import lru_cache
from pathlib import Path

from pydantic_settings import BaseSettings, SettingsConfigDict

BASE_DIR = Path(__file__).resolve().parent.parent


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

```

```
    database_url: str = f"sqlite:///{BASE_DIR / 'ysii.db'}"
    secret_key: str = "ganti-dengan-string-acak-panjang"
    token_expire_hours: int = 12

    # Domain website mitra yang boleh mengirim data tracking (dipisah koma)
    allowed_origins: str = "https://ysii.org,https://www.ysii.org"
    # URL publik API ini (dipakai tracker untuk memanggil balik)
    public_api_url: str = "https://api.ysii.org"
    # Nomor WhatsApp admin (format 62xxxx) untuk tombol "Hubungi" di dashboard
    whatsapp_admin: str = ""

    # Batas segmen skor konversi
    hot_threshold: float = 0.7
    warm_threshold: float = 0.4

    # Model ML
    model_dir: Path = BASE_DIR / "models"
    # Jumlah minimal contoh positif sebelum model ML menggantikan skor berbasis aturan
    min_positive_samples: int = 30

    # Google Search Console (opsional)
    gsc_site_url: str = "sc-domain:ysii.org"
    gsc_credentials_file: str = ""

    # Notifikasi lead "hot" (opsional, via webhook mis. n8n / Fonnte / Slack)
    hot_lead_webhook_url: str = ""

    @property
    def origins(self) -> list[str]:
        return [o.strip() for o in self.allowed_origins.split(",") if o.strip()]


@lru_cache
def get_settings() -> Settings:
    return Settings()
```

backend/app/db.py — Koneksi basis data (30 baris)

```
from collections.abc import Iterator

from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

from .config import get_settings


class Base(DeclarativeBase):
    pass


_settings = get_settings()
_connect_args = {"check_same_thread": False} if _settings.database_url.startswith("sqlite") el
se {}
engine = create_engine(_settings.database_url, connect_args=_connect_args, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)


def get_db() -> Iterator[Session]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()


def init_db() -> None:
    from . import models  # noqa: F401  (registrasi tabel)

```

```
    Base.metadata.create_all(bind=engine)
```

backend/app/models.py — Skema basis data (ORM) (171 baris)

```
"""Skema database (sesuai ERD di docs/LANGKAH-PEMBANGUNAN-SISTEM.md)."""

from datetime import date, datetime, timezone

from sqlalchemy import (
    JSON,
    Boolean,
    Date,
    DateTime,
    Float,
    ForeignKey,
    Index,
    Integer,
    String,
    Text,
)
from sqlalchemy.orm import Mapped, mapped_column, relationship

from .db import Base


def utcnow() -> datetime:
    return datetime.now(timezone.utc)


class Visitor(Base):
    __tablename__ = "visitors"

    visitor_id: Mapped[str] = mapped_column(String(36), primary_key=True)
    # Kode pendek yang ikut di pesan WhatsApp, agar chat WA bisa dihubungkan ke pengunjung
    ref_code: Mapped[str] = mapped_column(String(8), index=True)
    first_seen_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    last_seen_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    first_source: Mapped[str | None] = mapped_column(String(64))
    first_medium: Mapped[str | None] = mapped_column(String(32))
    first_campaign: Mapped[str | None] = mapped_column(String(128))
    first_landing: Mapped[str | None] = mapped_column(Text)
    device_type: Mapped[str | None] = mapped_column(String(16))

    sessions: Mapped[list["VisitSession"]] = relationship(back_populates="visitor")


class VisitSession(Base):
    __tablename__ = "sessions"

    session_id: Mapped[str] = mapped_column(String(36), primary_key=True)
    visitor_id: Mapped[str] = mapped_column(ForeignKey("visitors.visitor_id"), index=True)
    started_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    ended_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    source: Mapped[str | None] = mapped_column(String(64))
    medium: Mapped[str | None] = mapped_column(String(32))
    campaign: Mapped[str | None] = mapped_column(String(128))
    utm_term: Mapped[str | None] = mapped_column(String(256))
    landing_page: Mapped[str | None] = mapped_column(Text)
    referrer: Mapped[str | None] = mapped_column(Text)
    page_count: Mapped[int] = mapped_column(Integer, default=0)
    active_seconds: Mapped[int] = mapped_column(Integer, default=0)
    max_scroll_pct: Mapped[int] = mapped_column(Integer, default=0)

    visitor: Mapped[Visitor] = relationship(back_populates="sessions")


class Event(Base):
    __tablename__ = "events"
```

```
    __table_args__ = (Index("ix_events_visitor_ts", "visitor_id", "ts"),)

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    session_id: Mapped[str] = mapped_column(ForeignKey("sessions.session_id"))
    visitor_id: Mapped[str] = mapped_column(ForeignKey("visitors.visitor_id"))
    ts: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    # page_view, scroll, heartbeat, click_cta, click_whatsapp, view_price,
    # download_brochure, form_start, form_submit, site_search
    type: Mapped[str] = mapped_column(String(32), index=True)
    page_url: Mapped[str | None] = mapped_column(Text)
    page_category: Mapped[str | None] = mapped_column(String(32))
    program_slug: Mapped[str | None] = mapped_column(String(128), index=True)
    value: Mapped[float | None] = mapped_column(Float)
    label: Mapped[str | None] = mapped_column(String(256))

    # Relasi dipakai agar SQLAlchemy menyimpan visitor & session sebelum event (urutan FK)
    session: Mapped["VisitSession"] = relationship()
    visitor: Mapped["Visitor"] = relationship()


class Program(Base):
    __tablename__ = "programs"

    slug: Mapped[str] = mapped_column(String(128), primary_key=True)
    name: Mapped[str] = mapped_column(String(256))
    category: Mapped[str | None] = mapped_column(String(128))
    price: Mapped[float | None] = mapped_column(Float)
    url: Mapped[str | None] = mapped_column(Text)
    next_batch: Mapped[date | None] = mapped_column(Date)
    active: Mapped[bool] = mapped_column(Boolean, default=True)


LEAD_STATUSES = ["baru", "dihubungi", "tertarik", "terdaftar", "bayar", "batal"]
CONVERTED_STATUSES = {"terdaftar", "bayar"}


class Lead(Base):
    __tablename__ = "leads"

    lead_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    visitor_id: Mapped[str | None] = mapped_column(ForeignKey("visitors.visitor_id"), index=Tr
ue)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    channel: Mapped[str] = mapped_column(String(16), default="form")  # form / whatsapp
    name: Mapped[str | None] = mapped_column(String(256))
    phone: Mapped[str | None] = mapped_column(String(32))
    email: Mapped[str | None] = mapped_column(String(256))
    program_slug: Mapped[str | None] = mapped_column(String(128))
    message: Mapped[str | None] = mapped_column(Text)
    extra: Mapped[dict | None] = mapped_column(JSON)
    status: Mapped[str] = mapped_column(String(16), default="baru")
    notes: Mapped[str | None] = mapped_column(Text)
    # Label ML: True = menjadi peserta (terdaftar/bayar), False = batal, None = belum diketahu
i
    converted: Mapped[bool | None] = mapped_column(Boolean)
    converted_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    score: Mapped[float | None] = mapped_column(Float)
    segment: Mapped[str | None] = mapped_column(String(8))
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)

    visitor: Mapped[Visitor | None] = relationship()


class Prediction(Base):
    __tablename__ = "predictions"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    visitor_id: Mapped[str] = mapped_column(String(36), index=True)
```

```
    lead_id: Mapped[int | None] = mapped_column(Integer)
    model_version: Mapped[str] = mapped_column(String(80))
    score: Mapped[float] = mapped_column(Float)
    segment: Mapped[str] = mapped_column(String(8))
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)


class GscDaily(Base):
    __tablename__ = "gsc_daily"

    date: Mapped[date] = mapped_column(Date, primary_key=True)
    page: Mapped[str] = mapped_column(String(512), primary_key=True)
    query: Mapped[str] = mapped_column(String(512), primary_key=True)
    clicks: Mapped[int] = mapped_column(Integer)
    impressions: Mapped[int] = mapped_column(Integer)
    ctr: Mapped[float] = mapped_column(Float)
    position: Mapped[float] = mapped_column(Float)


class ModelRegistry(Base):
    __tablename__ = "model_registry"

    version: Mapped[str] = mapped_column(String(80), primary_key=True)
    # "lead" (lead -> peserta) atau "visitor" (pengunjung -> lead)
    task: Mapped[str] = mapped_column(String(16), default="lead", index=True)
    algorithm: Mapped[str] = mapped_column(String(64))
    trained_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    n_samples: Mapped[int] = mapped_column(Integer)
    n_positive: Mapped[int] = mapped_column(Integer)
    metrics: Mapped[dict] = mapped_column(JSON)
    is_active: Mapped[bool] = mapped_column(Boolean, default=False)
    artifact_path: Mapped[str] = mapped_column(Text)


class AdminUser(Base):
    __tablename__ = "admin_users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    username: Mapped[str] = mapped_column(String(64), unique=True)
    password_hash: Mapped[str] = mapped_column(String(256))
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
```

backend/app/security.py — Hash password & token login (63 baris)

```
import base64
import hashlib
import hmac
import secrets
from datetime import datetime, timedelta, timezone

import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy import select
from sqlalchemy.orm import Session

from .config import get_settings
from .db import get_db
from .models import AdminUser

_ITERATIONS = 310_000
_bearer = HTTPBearer(auto_error=False)


def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, _ITERATIONS)
    return "pbkdf2_sha256${}${}${}".format(
```

```
        _ITERATIONS, base64.b64encode(salt).decode(), base64.b64encode(digest).decode()
    )


def verify_password(password: str, stored: str) -> bool:
    try:
        _, iterations, salt_b64, digest_b64 = stored.split("$")
    except ValueError:
        return False
    digest = hashlib.pbkdf2_hmac(
        "sha256", password.encode(), base64.b64decode(salt_b64), int(iterations)
    )
    return hmac.compare_digest(digest, base64.b64decode(digest_b64))


def create_token(username: str) -> str:
    s = get_settings()
    payload = {
        "sub": username,
        "exp": datetime.now(timezone.utc) + timedelta(hours=s.token_expire_hours),
    }
    return jwt.encode(payload, s.secret_key, algorithm="HS256")


def current_admin(
    creds: HTTPAuthorizationCredentials | None = Depends(_bearer),
    db: Session = Depends(get_db),
) -> AdminUser:
    unauthorized = HTTPException(status.HTTP_401_UNAUTHORIZED, "Silakan login kembali")
    if creds is None:
        raise unauthorized
    try:
        payload = jwt.decode(creds.credentials, get_settings().secret_key, algorithms=["HS25
6"])
    except jwt.PyJWTError:
        raise unauthorized
    user = db.scalar(select(AdminUser).where(AdminUser.username == payload.get("sub")))
    if user is None or not user.is_active:
        raise unauthorized
    return user
```

backend/app/traffic.py — Klasifikasi sumber traffic & intent (96 baris)

```
"""Klasifikasi sumber traffic, kategori halaman, dan intent kata kunci."""

import re
from urllib.parse import urlparse

SEARCH_ENGINES = ("google.", "bing.", "yahoo.", "duckduckgo.", "yandex.", "ecosia.")
SOCIAL = {
    "facebook": ("facebook.", "fb.", "m.facebook"),
    "instagram": ("instagram.",),
    "tiktok": ("tiktok.",),
    "youtube": ("youtube.", "youtu.be"),
    "linkedin": ("linkedin.", "lnkd.in"),
    "twitter": ("twitter.", "t.co", "x.com"),
    "whatsapp": ("whatsapp.", "wa.me"),
    "telegram": ("t.me", "telegram."),
}
PAID_MEDIUMS = {"cpc", "ppc", "paid", "paidsearch", "paid_social", "ads"}


def classify_source(
    referrer: str | None,
    utm_source: str | None,
    utm_medium: str | None,
    own_hosts: tuple[str, ...] = ("ysii.org",),
) -> tuple[str, str]:
```

```
    """Kembalikan (source, medium), misal ('google', 'organic')."""
    if utm_source:
        medium = (utm_medium or "referral").lower()
        if medium in PAID_MEDIUMS:
            medium = "cpc"
        return utm_source.lower()[:64], medium[:32]

    host = (urlparse(referrer).hostname or "").lower() if referrer else ""
    if not host or any(host.endswith(h) for h in own_hosts):
        return "direct", "none"
    for engine in SEARCH_ENGINES:
        if engine in host:
            return engine.rstrip("."), "organic"
    for name, patterns in SOCIAL.items():
        if any(p in host for p in patterns):
            return name, "social"
    return host[:64], "referral"


def page_category(path: str | None, program_slug: str | None = None) -> str:
    if program_slug:
        return "program"
    p = (path or "/").lower()
    if p in ("", "/", "/index.html", "/home"):
        return "beranda"
    if re.search(r"/(program|pelatihan|kursus|training|sertifikasi|skema|kelas)", p):
        return "program"
    if re.search(r"/(daftar|pendaftaran|register|registrasi)", p):
        return "pendaftaran"
    if re.search(r"/(kontak|contact|hubungi)", p):
        return "kontak"
    if re.search(r"/(tentang|about|profil|visi)", p):
        return "tentang"
    if re.search(r"/(blog|artikel|berita|news|\d{4}/\d{2})", p):
        return "artikel"
    return "lainnya"


_INTENT_RULES = [
    ("brand", r"\b(ysii|sinergi inspirasi)\b"),
    ("transaksional", r"\b(daftar|pendaftaran|biaya|harga|jadwal|registrasi|promo|diskon)\b"),
    ("lokal", r"\b(medan|sumut|sumatera utara|terdekat|dekat)\b"),
    ("komersial", r"\b(pelatihan|kursus|sertifikasi|training|bnsp|kelas|bersertifikat|lsp)
\b"),
    ("informasional", r"\b(apa|cara|adalah|bagaimana|manfaat|contoh|pengertian|tips|prospek)
\b"),
]


def query_intent(query: str | None) -> str:
    """Kategori intent kata kunci pencarian (aturan sederhana, bisa diperluas)."""
    if not query:
        return "tidak_diketahui"
    q = query.lower()
    for intent, pattern in _INTENT_RULES:
        if re.search(pattern, q):
            return intent
    return "lainnya"


def normalize_path(url: str | None) -> str:
    if not url:
        return "/"
    parsed = urlparse(url)
    return (parsed.path or "/").rstrip("/") or "/"


```

```
_PROGRAM_PATH = re.compile(r"/(?:program|pelatihan|kursus|sertifikasi|training|kelas)/([a-z0-9-]+)", re.I)


def program_from_path(path: str | None) -> str | None:
    m = _PROGRAM_PATH.search(path or "")
    return m.group(1).lower() if m else None
```

backend/app/features.py — Feature engineering perilaku (190 baris)

```
"""Feature engineering perilaku pengunjung (dipakai bersama oleh training & scoring).

Satu baris fitur = satu pengunjung, dihitung dari event-nya sampai waktu `as_of`.
Untuk data latih, `as_of` = saat pengunjung menjadi lead, supaya model hanya belajar
dari perilaku SEBELUM mendaftar (mencegah kebocoran label).
"""

from collections import defaultdict
from collections.abc import Iterable
from datetime import datetime, timezone

from sqlalchemy import func, select
from sqlalchemy.orm import Session

from .models import Event, GscDaily, Visitor
from .traffic import normalize_path, page_category, query_intent

NUMERIC_FEATURES = [
    "n_sessions",
    "n_pageviews",
    "total_active_sec",
    "avg_active_per_session",
    "avg_pages_per_session",
    "max_scroll_pct",
    "avg_session_scroll_pct",
    "bounce_rate",
    "n_program_views",
    "n_distinct_programs",
    "n_article_views",
    "n_deep_program_reads",
    "n_click_cta",
    "n_click_whatsapp",
    "n_view_price",
    "n_download_brochure",
    "n_form_start",
    "n_site_search",
    "is_returning",
    "n_active_days",
    "engagement_span_days",
    "first_visit_hour",
    "first_visit_weekend",
]
CATEGORICAL_FEATURES = [
    "first_medium",
    "first_source",
    "device_type",
    "landing_category",
    "landing_query_intent",
]
ALL_FEATURES = NUMERIC_FEATURES + CATEGORICAL_FEATURES

# Fitur yang dikeluarkan per tugas. Untuk "pengunjung -> lead", klik WhatsApp dan mulai
# mengisi form adalah aksi menjadi lead itu sendiri, jadi memakainya = membocorkan label.
_EXCLUDED = {"visitor": {"n_click_whatsapp", "n_form_start"}, "lead": set()}


def task_features(task: str) -> tuple[list[str], list[str]]:
    excluded = _EXCLUDED[task]
```

```
    return (
        [f for f in NUMERIC_FEATURES if f not in excluded],
        [f for f in CATEGORICAL_FEATURES if f not in excluded],
    )

_COUNTED_EVENTS = {
    "click_cta": "n_click_cta",
    "click_whatsapp": "n_click_whatsapp",
    "view_price": "n_view_price",
    "download_brochure": "n_download_brochure",
    "form_start": "n_form_start",
    "site_search": "n_site_search",
}
_TOP_SOURCES = {"google", "direct", "facebook", "instagram", "tiktok", "youtube", "whatsapp"}


def _aware(dt: datetime) -> datetime:
    return dt if dt.tzinfo else dt.replace(tzinfo=timezone.utc)


def compute_features(
    visitor: Visitor,
    events: Iterable[Event],
    landing_intents: dict[str, str],
    as_of: datetime | None = None,
) -> dict:
    cutoff = _aware(as_of) if as_of else None
    evs = sorted(
        (e for e in events if cutoff is None or _aware(e.ts) <= cutoff),
        key=lambda e: _aware(e.ts),
    )

    f: dict = {k: 0 for k in NUMERIC_FEATURES}
    per_session_pages: dict[str, int] = defaultdict(int)
    per_session_active: dict[str, float] = defaultdict(float)
    per_session_scroll: dict[str, float] = defaultdict(float)
    programs: set[str] = set()
    days: set = set()

    for e in evs:
        sid = e.session_id
        per_session_pages.setdefault(sid, 0)
        days.add(_aware(e.ts).date())
        if e.type == "page_view":
            per_session_pages[sid] += 1
            f["n_pageviews"] += 1
            if e.page_category == "program":
                f["n_program_views"] += 1
                if e.program_slug:
                    programs.add(e.program_slug)
            elif e.page_category == "artikel":
                f["n_article_views"] += 1
        elif e.type == "heartbeat":
            per_session_active[sid] += max(0.0, min(float(e.value or 0), 1800.0))
        elif e.type == "scroll":
            v = float(e.value or 0)
            per_session_scroll[sid] = max(per_session_scroll[sid], v)
            if v >= 75 and e.page_category == "program":
                f["n_deep_program_reads"] += 1
        elif e.type in _COUNTED_EVENTS:
            f[_COUNTED_EVENTS[e.type]] += 1

    sessions = list(per_session_pages)
    n_sessions = len(sessions)
    f["n_sessions"] = n_sessions
    f["n_distinct_programs"] = len(programs)
    f["total_active_sec"] = round(sum(per_session_active.values()), 1)
    if n_sessions:
```

```
        f["avg_active_per_session"] = round(f["total_active_sec"] / n_sessions, 1)
        f["avg_pages_per_session"] = round(f["n_pageviews"] / n_sessions, 2)
        f["max_scroll_pct"] = max((per_session_scroll.get(s, 0) for s in sessions), default=0)
        f["avg_session_scroll_pct"] = round(
            sum(per_session_scroll.get(s, 0) for s in sessions) / n_sessions, 1
        )
        bounces = sum(
            1
            for s in sessions
            if per_session_pages[s] <= 1 and per_session_active.get(s, 0) < 10
        )
        f["bounce_rate"] = round(bounces / n_sessions, 3)
    f["is_returning"] = int(n_sessions > 1)
    f["n_active_days"] = len(days)

    # Rentang hari dari kunjungan pertama sampai aktivitas terakhir (sebelum cutoff).
    # Sengaja TIDAK memakai "hari sejak kunjungan terakhir sampai sekarang": nilai itu
    # berbeda cara hitungnya antara lead (dipotong saat mendaftar) dan non-lead,
    # sehingga membocorkan label ke model.
    first = _aware(visitor.first_seen_at)
    last_ts = _aware(evs[-1].ts) if evs else first
    f["engagement_span_days"] = round(max(0.0, (last_ts - first).total_seconds() / 86400), 2)
    f["first_visit_hour"] = first.hour
    f["first_visit_weekend"] = int(first.weekday() >= 5)

    source = (visitor.first_source or "direct").lower()
    landing_path = normalize_path(visitor.first_landing)
    f["first_medium"] = visitor.first_medium or "none"
    f["first_source"] = source if source in _TOP_SOURCES else "lainnya"
    f["device_type"] = visitor.device_type or "unknown"
    f["landing_category"] = page_category(landing_path)
    f["landing_query_intent"] = landing_intents.get(landing_path, "tidak_diketahui")
    return f


def landing_intents(db: Session, paths: Iterable[str] | None = None) -> dict[str, str]:
    """Intent kata kunci dominan (berdasar klik GSC) untuk setiap landing page.

    Google tidak memberikan keyword per pengunjung, jadi keyword dominan dari
    halaman pendaratan dipakai sebagai proksi intent pencarian.
    """
    stmt = select(GscDaily.page, GscDaily.query, func.sum(GscDaily.clicks)).group_by(
        GscDaily.page, GscDaily.query
    )
    best: dict[str, tuple[str, int]] = {}
    wanted = set(paths) if paths is not None else None
    for page, query, clicks in db.execute(stmt):
        path = normalize_path(page)
        if wanted is not None and path not in wanted:
            continue
        if path not in best or (clicks or 0) > best[path][1]:
            best[path] = (query, clicks or 0)
    return {p: query_intent(q) for p, (q, _) in best.items()}


def visitor_features(
    db: Session, visitor_id: str, as_of: datetime | None = None
) -> dict | None:
    visitor = db.get(Visitor, visitor_id)
    if visitor is None:
        return None
    events = db.scalars(select(Event).where(Event.visitor_id == visitor_id)).all()
    intents = landing_intents(db, [normalize_path(visitor.first_landing)])
    return compute_features(visitor, events, intents, as_of=as_of)
```

backend/app/ml_engine.py — Pelatihan & perbandingan algoritma ML (287 baris)

```
"""ML Predictive Engine — pelatihan (CRISP-DM) dan perbandingan tiga algoritma.
```

```
 2
 3 Ada dua tugas prediksi dengan model terpisah:
 4
 5 - task="lead"    : lead -> menjadi peserta (terdaftar/bayar).
 6                    Dilatih HANYA dari lead yang hasilnya sudah diketahui, sehingga skor
 7                    benar-benar membedakan lead serius dari yang tidak. Dipakai untuk
 8                    memprioritaskan lead di dashboard.
 9 - task="visitor" : pengunjung -> menjadi lead (isi form / chat WhatsApp).
10                    Dilatih dari semua pengunjung. Dipakai untuk rekomendasi konten & CTA.
11
12 Logistic Regression (baseline), Random Forest, dan Gradient Boosting dibandingkan
13 dengan stratified k-fold cross-validation. Model terbaik (berdasar F1) disimpan
14 dan diaktifkan.
15 """
16
17 from __future__ import annotations
18
19 import logging
20 import threading
21 from collections import defaultdict
22 from datetime import datetime, timedelta, timezone
23
24 import joblib
25 import numpy as np
26 import pandas as pd
27 from sklearn.compose import ColumnTransformer
28 from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
29 from sklearn.inspection import permutation_importance
30 from sklearn.linear_model import LogisticRegression
31 from sklearn.metrics import (
32     accuracy_score,
33     confusion_matrix,
34     f1_score,
35     precision_score,
36     recall_score,
37     roc_auc_score,
38 )
39 from sklearn.model_selection import StratifiedKFold, cross_validate, train_test_split
40 from sklearn.pipeline import Pipeline
41 from sklearn.preprocessing import OneHotEncoder, StandardScaler
42 from sqlalchemy import select, update
43 from sqlalchemy.orm import Session
44
45 from .config import get_settings
46 from .features import compute_features, landing_intents, task_features
47 from .models import Event, Lead, ModelRegistry, Visitor
48
49 log = logging.getLogger(__name__)
50 _train_lock = threading.Lock()
51
52 TASKS = {
53     "lead": "Lead → menjadi peserta (prioritas lead)",
54     "visitor": "Pengunjung → menjadi lead (rekomendasi & CTA)",
55 }
56
57 ALGORITHMS = {
58     "logistic_regression": lambda: LogisticRegression(max_iter=2000, class_weight="balanced"),
59     "random_forest": lambda: RandomForestClassifier(
60         n_estimators=300, min_samples_leaf=2, class_weight="balanced", random_state=42, n_jobs
   =-1
61     ),
62     "gradient_boosting": lambda: HistGradientBoostingClassifier(
63         max_iter=300, learning_rate=0.05, class_weight="balanced", random_state=42
64     ),
65 }
66
67 OPEN_STATUSES = ("baru", "dihubungi", "tertarik")
68
```

```

class InsufficientDataError(Exception):
    pass


def _aware(dt: datetime) -> datetime:
    return dt if dt.tzinfo else dt.replace(tzinfo=timezone.utc)


def build_dataset(
    db: Session, task: str = "lead", negative_min_idle_days: float = 1.0
) -> pd.DataFrame:
    """Satu baris per pengunjung + kolom `y`.

    Fitur selalu dihitung sampai saat pengunjung pertama kali menjadi lead (as_of),
    supaya model hanya belajar dari perilaku SEBELUM mendaftar.
    """
    if task not in TASKS:
        raise ValueError(f"task harus salah satu dari {list(TASKS)}")
    now = datetime.now(timezone.utc)
    leads_by_v: dict[str, list[Lead]] = defaultdict(list)
    for lead in db.scalars(select(Lead).where(Lead.visitor_id.is_not(None))):
        leads_by_v[lead.visitor_id].append(lead)

    vq = select(Visitor)
    eq = select(Event)
    if task == "lead":
        vq = vq.where(Visitor.visitor_id.in_(list(leads_by_v)))
        eq = eq.where(Event.visitor_id.in_(list(leads_by_v)))
    visitors = db.scalars(vq).all()
    events_by_v: dict[str, list[Event]] = defaultdict(list)
    for e in db.scalars(eq):
        events_by_v[e.visitor_id].append(e)
    intents = landing_intents(db)

    rows = []
    for v in visitors:
        leads = leads_by_v.get(v.visitor_id, [])
        if leads:
            as_of = min(_aware(l.created_at) for l in leads)
            if task == "visitor":
                y = 1
            elif any(l.converted for l in leads):
                y = 1
            elif all(l.converted is False for l in leads):
                y = 0
            else:
                continue  # hasil follow-up belum diketahui
        else:
            if (now - _aware(v.last_seen_at)) < timedelta(days=negative_min_idle_days):
                continue  # masih mungkin mendaftar
            as_of, y = None, 0
        feats = compute_features(v, events_by_v.get(v.visitor_id, []), intents, as_of=as_of)
        if feats["n_sessions"] == 0:
            continue
        feats["visitor_id"] = v.visitor_id
        feats["y"] = y
        rows.append(feats)
    return pd.DataFrame(rows)


def _pipeline(algorithm: str, task: str) -> Pipeline:
    numeric, categorical = task_features(task)
    prep = ColumnTransformer(
        [
            ("num", StandardScaler(), numeric),
            (
                "cat",
```

```
                OneHotEncoder(handle_unknown="ignore", sparse_output=False, min_frequency=3),
                categorical,
            ),
        ]
    )
    return Pipeline([("prep", prep), ("clf", ALGORITHMS[algorithm]())])


def _metrics(y_true, proba, threshold: float = 0.5) -> dict:
    pred = (proba >= threshold).astype(int)
    out = {
        "accuracy": accuracy_score(y_true, pred),
        "precision": precision_score(y_true, pred, zero_division=0),
        "recall": recall_score(y_true, pred, zero_division=0),
        "f1": f1_score(y_true, pred, zero_division=0),
    }
    if len(set(y_true)) > 1:
        out["roc_auc"] = roc_auc_score(y_true, proba)
    out = {k: round(float(v), 4) for k, v in out.items()}
    out["confusion_matrix"] = confusion_matrix(y_true, pred, labels=[0, 1]).tolist()
    return out


def train_models(
    db: Session, task: str = "lead", force: bool = False, activate: bool = True
) -> dict:
    """Latih & bandingkan 3 algoritma untuk satu tugas, simpan model terbaik."""
    if not _train_lock.acquire(blocking=False):
        raise RuntimeError("Pelatihan model sedang berjalan")
    try:
        return _train(db, task, force, activate)
    finally:
        _train_lock.release()


def train_all(db: Session, force: bool = False) -> dict[str, dict]:
    """Latih semua tugas yang datanya cukup. Kembalikan {task: laporan | {"error": ...}}."""
    results: dict[str, dict] = {}
    for task in TASKS:
        try:
            results[task] = train_models(db, task=task, force=force)
        except InsufficientDataError as exc:
            results[task] = {"error": str(exc)}
    return results


def _train(db: Session, task: str, force: bool, activate: bool) -> dict:
    s = get_settings()
    df = build_dataset(db, task)
    n_pos = int(df["y"].sum()) if len(df) else 0
    n_neg = len(df) - n_pos
    label = TASKS[task]
    if n_pos < 5 or n_neg < 5:
        raise InsufficientDataError(
            f"[{label}] data belum cukup: {n_pos} positif, {n_neg} negatif (minimal 5 dan 5)"
        )
    if min(n_pos, n_neg) < s.min_positive_samples and not force:
        raise InsufficientDataError(
            f"[{label}] baru {n_pos} positif & {n_neg} negatif; butuh minimal "
            f"{s.min_positive_samples} untuk masing-masing. Sistem tetap memakai skor berbasis
aturan."
        )

    numeric, categorical = task_features(task)
    X = df[numeric + categorical]
    y = df["y"].to_numpy()
    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=4
2)
```

```
    n_splits = int(max(2, min(5, y_tr.sum(), len(y_tr) - y_tr.sum())))
    cv = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)

    comparison: dict[str, dict] = {}
    for name in ALGORITHMS:
        scores = cross_validate(
            _pipeline(name, task), X_tr, y_tr, cv=cv,
            scoring=["accuracy", "precision", "recall", "f1", "roc_auc"],
        )
        cv_res = {}
        for k, v in scores.items():
            if k.startswith("test_"):
                cv_res[k[5:]] = round(float(np.mean(v)), 4)
                cv_res[k[5:] + "_std"] = round(float(np.std(v)), 4)
        fitted = _pipeline(name, task).fit(X_tr, y_tr)
        comparison[name] = {"cv": cv_res, "holdout": _metrics(y_te, fitted.predict_proba(X_te)
[:, 1])}

    best = max(
        comparison,
        key=lambda n: (comparison[n]["cv"]["f1"], comparison[n]["cv"].get("roc_auc", 0)),
    )

    # Kepentingan fitur (permutation importance) pada data uji, untuk interpretasi
    best_holdout = _pipeline(best, task).fit(X_tr, y_tr)
    imp = permutation_importance(
        best_holdout, X_te, y_te, scoring="roc_auc", n_repeats=5, random_state=42
    )
    importance = sorted(
        (
            {"feature": f, "importance": round(float(m), 4)}
            for f, m in zip(X.columns, imp.importances_mean)
        ),
        key=lambda d: d["importance"],
        reverse=True,
    )

    final = _pipeline(best, task).fit(X, y)  # model produksi dilatih dengan seluruh data
    version = f"{task}-{best}-{datetime.now(timezone.utc):%Y%m%d%H%M%S}"
    s.model_dir.mkdir(parents=True, exist_ok=True)
    path = s.model_dir / f"{version}.joblib"
    joblib.dump(
        {"pipeline": final, "version": version, "task": task, "features": list(X.columns)}, pa
th
    )

    report = {
        "version": version,
        "task": task,
        "task_label": label,
        "best_algorithm": best,
        "n_samples": int(len(df)),
        "n_positive": n_pos,
        "cv_folds": n_splits,
        "comparison": comparison,
        "feature_importance": importance,
    }
    if activate:
        db.execute(update(ModelRegistry).where(ModelRegistry.task == task).values(is_active=Fa
lse))
    db.add(
        ModelRegistry(
            version=version, task=task, algorithm=best, n_samples=len(df), n_positive=n_pos,
            metrics=report, is_active=activate, artifact_path=str(path),
        )
    )
    db.commit()
    if activate and task == "lead":
```

```
        report["rescored_leads"] = rescore_open_leads(db)
    log.info("Model %s dilatih (F1 CV=%.3f)", version, comparison[best]["cv"]["f1"])
    return report


def rescore_open_leads(db: Session) -> int:
    """Hitung ulang skor lead yang masih dalam proses follow-up dengan model aktif.

    Lead yang sudah selesai (terdaftar/bayar/batal) tidak diubah, agar skor aslinya
    tetap bisa dipakai untuk mengevaluasi ketepatan prediksi.
    """
    from .routers.public import score_lead  # impor lokal: hindari impor melingkar

    leads = db.scalars(
        select(Lead).where(Lead.status.in_(OPEN_STATUSES), Lead.visitor_id.is_not(None))
    ).all()
    for lead in leads:
        score_lead(db, lead)
    db.commit()
    return len(leads)
```

backend/app/scoring.py — Skor konversi & segmentasi (149 baris)

```
"""Scoring probabilitas + segmentasi (Hot / Warm / Cold).

task="lead"    : peluang lead menjadi peserta (prioritas follow-up di dashboard)
task="visitor" : peluang pengunjung menjadi lead (rekomendasi & CTA di website)

Sebelum data cukup untuk model ML, dipakai skor berbasis aturan.
"""

from __future__ import annotations

import math
import threading

import joblib
import pandas as pd
from sqlalchemy import select
from sqlalchemy.orm import Session

from .config import get_settings
from .features import task_features
from .models import ModelRegistry

RULE_VERSION = "rules-v1"

# (fitur, bobot, batas nilai, alasan) — bobot dalam skala logit
_RULES_VISITOR = [
    ("n_click_whatsapp", 1.2, 1, "Klik tombol WhatsApp"),
    ("n_form_start", 1.0, 1, "Mulai mengisi form pendaftaran"),
    ("n_view_price", 0.7, 2, "Melihat biaya/jadwal"),
    ("n_download_brochure", 0.6, 1, "Mengunduh brosur"),
    ("n_click_cta", 0.5, 2, "Klik tombol ajakan (CTA)"),
    ("n_deep_program_reads", 0.4, 3, "Membaca halaman program sampai bawah"),
    ("n_distinct_programs", 0.3, 3, "Melihat beberapa program"),
    ("is_returning", 0.8, 1, "Pengunjung kembali"),
    ("n_active_days", 0.3, 3, "Aktif di beberapa hari berbeda"),
]
# Untuk lead, mengisi form / chat WA sudah pasti terjadi, jadi tidak dihitung.
# Yang membedakan lead serius adalah kedalaman riset sebelum mendaftar.
_RULES_LEAD = [
    ("n_view_price", 0.6, 2, "Melihat biaya/jadwal"),
    ("n_download_brochure", 0.5, 1, "Mengunduh brosur"),
    ("n_deep_program_reads", 0.4, 3, "Membaca halaman program sampai bawah"),
    ("n_program_views", 0.15, 5, "Sering membuka halaman program"),
    ("is_returning", 0.6, 1, "Pengunjung kembali"),
    ("n_active_days", 0.25, 3, "Aktif di beberapa hari berbeda"),
```

```
    ("n_click_cta", 0.3, 2, "Klik tombol ajakan (CTA)"),
]
_RULES = {"visitor": (_RULES_VISITOR, -3.0), "lead": (_RULES_LEAD, -1.6)}

_cache_lock = threading.Lock()
_cache: dict[str, dict] = {}


def segment_for(score: float) -> str:
    s = get_settings()
    if score >= s.hot_threshold:
        return "hot"
    if score >= s.warm_threshold:
        return "warm"
    return "cold"


def rule_score(f: dict, task: str = "lead") -> tuple[float, list[str]]:
    rules, z = _RULES[task]
    reasons: list[tuple[float, str]] = []
    for key, weight, cap, text in rules:
        v = min(float(f.get(key, 0) or 0), cap)
        if v > 0:
            z += weight * v
            reasons.append((weight * v, text))
    if f.get("total_active_sec", 0) >= 120:
        z += 0.5
        reasons.append((0.5, "Waktu baca lebih dari 2 menit"))
    if f.get("first_medium") == "organic":
        z += 0.2
        reasons.append((0.2, "Datang dari pencarian Google"))
    if f.get("bounce_rate", 0) >= 1:
        z -= 0.5
    reasons.sort(reverse=True)
    return 1 / (1 + math.exp(-z)), [r for _, r in reasons[:4]]


def _active_model(db: Session, task: str):
    reg = db.scalar(
        select(ModelRegistry).where(ModelRegistry.is_active.is_(True), ModelRegistry.task == t
ask)
    )
    if reg is None:
        return None, None, []
    with _cache_lock:
        cached = _cache.get(task)
        if cached is None or cached["version"] != reg.version:
            try:
                artifact = joblib.load(reg.artifact_path)
            except (OSError, EOFError):
                return None, None, []
            cached = {
                "version": reg.version,
                "pipeline": artifact["pipeline"],
                "importance": reg.metrics.get("feature_importance", []),
            }
            _cache[task] = cached
        return cached["version"], cached["pipeline"], cached["importance"]


_FEATURE_TEXT = {key: text for key, _, _, text in _RULES_VISITOR + _RULES_LEAD} | {
    "total_active_sec": "Waktu baca lama",
    "avg_active_per_session": "Waktu baca lama",
    "n_pageviews": "Banyak halaman dibuka",
    "avg_pages_per_session": "Banyak halaman dibuka",
    "max_scroll_pct": "Membaca halaman sampai bawah",
    "n_sessions": "Beberapa kali berkunjung",
}
```

```
# Nilai minimal agar sebuah fitur layak disebut sebagai alasan
_REASON_MIN = {
    "n_distinct_programs": 2, "n_sessions": 2, "n_active_days": 2, "max_scroll_pct": 75,
    "total_active_sec": 120, "avg_active_per_session": 90, "n_pageviews": 3,
    "avg_pages_per_session": 3, "n_program_views": 2,
}


def _ml_reasons(f: dict, importance: list[dict]) -> list[str]:
    out: list[str] = []
    for item in importance:
        key = item["feature"]
        text = _FEATURE_TEXT.get(key)
        if item["importance"] > 0 and text and text not in out and (f.get(key) or 0) >= _REASO
N_MIN.get(key, 1):
            out.append(text)
        if len(out) == 4:
            break
    return out


def score_features(db: Session, f: dict, task: str = "lead") -> dict:
    version, pipeline, importance = _active_model(db, task)
    if pipeline is not None:
        numeric, categorical = task_features(task)
        X = pd.DataFrame([{k: f[k] for k in numeric + categorical}])
        p = float(pipeline.predict_proba(X)[:, 1][0])
        reasons = _ml_reasons(f, importance)
    else:
        version = RULE_VERSION
        p, reasons = rule_score(f, task)
    return {
        "score": round(p, 4),
        "segment": segment_for(p),
        "model_version": version,
        "task": task,
        "reasons": reasons,
    }
```

backend/app/recommend.py — Rekomendasi konten (86 baris)

```
"""Modul rekomendasi konten dinamis.

- Program: gabungan minat pengunjung sendiri + "pengunjung lain yang melihat program ini
  juga melihat ..." (item co-occurrence), fallback ke program dengan batch terdekat.
- CTA: disesuaikan dengan segmen skor konversi pengunjung.
"""

from collections import Counter, defaultdict
from datetime import date, datetime, timedelta, timezone

from sqlalchemy import select
from sqlalchemy.orm import Session

from .models import Event, Program

CTA_BY_SEGMENT = {
    "hot": {
        "type": "daftar",
        "title": "Kuota batch berikutnya terbatas",
        "text": "Amankan kursi Anda sekarang, atau konsultasi langsung dengan admin via WhatsA
pp.",
        "button": "Daftar sekarang",
    },
    "warm": {
        "type": "konsultasi",
        "title": "Masih ragu memilih program?",
        "text": "Lihat jadwal batch terdekat dan cerita alumni, atau tanya admin kami dulu.",
```

```
        "button": "Konsultasi gratis via WhatsApp",
    },
    "cold": {
        "type": "edukasi",
        "title": "Kenali program pelatihan kami",
        "text": "Unduh brosur untuk melihat materi, sertifikat, dan prospek kerja setiap program.",
        "button": "Lihat program",
    },
}


def _program_dict(p: Program) -> dict:
    return {
        "slug": p.slug,
        "name": p.name,
        "category": p.category,
        "url": p.url,
        "price": p.price,
        "next_batch": p.next_batch.isoformat() if p.next_batch else None,
    }


def recommend_programs(
    db: Session, visitor_id: str, current: str | None = None, limit: int = 3
) -> list[dict]:
    programs = {p.slug: p for p in db.scalars(select(Program).where(Program.active.is_(True)))}
    if not programs:
        return []

    since = datetime.now(timezone.utc) - timedelta(days=90)
    rows = db.execute(
        select(Event.visitor_id, Event.program_slug).where(
            Event.type == "page_view", Event.program_slug.is_not(None), Event.ts >= since
        )
    ).all()
    seen_by: dict[str, set[str]] = defaultdict(set)
    for vid, slug in rows:
        seen_by[vid].add(slug)

    mine = seen_by.get(visitor_id, set())
    if current:
        mine = mine | {current}

    scores: Counter = Counter()
    for vid, slugs in seen_by.items():
        if vid == visitor_id or not (slugs & mine):
            continue
        for s in slugs - mine:
            scores[s] += len(slugs & mine)

    ranked = [s for s, _ in scores.most_common() if s in programs]
    # Program yang pernah dilihat tapi bukan halaman yang sedang dibuka juga relevan
    ranked += [s for s in mine if s in programs and s != current and s not in ranked]
    today = date.today()
    upcoming = sorted(
        (p for p in programs.values() if p.slug not in ranked and p.slug != current),
        key=lambda p: (p.next_batch is None or p.next_batch < today, p.next_batch or date.max),
    )
    ranked += [p.slug for p in upcoming]
    return [_program_dict(programs[s]) for s in ranked[:limit]]
```

backend/app/routers/public.py — API untuk website (tracking, lead, rekomendasi) (375 baris)

```
"""Endpoint publik yang dipanggil dari website ysii.org (tracker.js & form)."""

```

```python
import json
import logging
import re
import threading
import time
import uuid
from collections import defaultdict, deque
from datetime import datetime, timedelta, timezone
from urllib.parse import urlparse

import httpx
from fastapi import APIRouter, BackgroundTasks, Depends, HTTPException, Request, Response
from fastapi.responses import RedirectResponse
from pydantic import BaseModel, Field, ValidationError
from sqlalchemy import select
from sqlalchemy.orm import Session

from ..config import get_settings
from ..db import SessionLocal, get_db
from ..features import visitor_features
from ..models import Event, Lead, Prediction, Visitor, VisitSession, utcnow
from ..recommend import CTA_BY_SEGMENT, recommend_programs
from ..scoring import score_features
from ..traffic import classify_source, normalize_path, page_category, program_from_path

log = logging.getLogger(__name__)
router = APIRouter(prefix="/v1", tags=["publik"])

ALLOWED_EVENTS = {
    "page_view", "scroll", "heartbeat", "click_cta", "click_whatsapp", "view_price",
    "download_brochure", "form_start", "form_submit", "site_search",
}

# ---------- rate limit sederhana per IP (per proses) ----------
_hits: dict[str, deque] = defaultdict(deque)
_hits_lock = threading.Lock()


def _rate_limited(ip: str, limit: int, window: float = 60.0) -> bool:
    now = time.monotonic()
    with _hits_lock:
        q = _hits[ip]
        while q and now - q[0] > window:
            q.popleft()
        if len(q) >= limit:
            return True
        q.append(now)
        return False


def _client_ip(request: Request) -> str:
    fwd = request.headers.get("x-forwarded-for")
    return fwd.split(",")[0].strip() if fwd else (request.client.host if request.client else "?")


def ref_code_for(visitor_id: str) -> str:
    return visitor_id.replace("-", "")[:6].upper()


# ---------- /collect ----------
class Ctx(BaseModel):
    visitor_id: uuid.UUID
    session_id: uuid.UUID
    page_url: str = Field(max_length=2000)
    page_title: str | None = Field(default=None, max_length=300)
    referrer: str | None = Field(default=None, max_length=2000)
    utm_source: str | None = Field(default=None, max_length=128)
```

```
    utm_medium: str | None = Field(default=None, max_length=64)
    utm_campaign: str | None = Field(default=None, max_length=128)
    utm_term: str | None = Field(default=None, max_length=256)
    program_slug: str | None = Field(default=None, max_length=128)
    device: str | None = Field(default=None, max_length=16)


class EventIn(BaseModel):
    type: str = Field(max_length=32)
    ts: int | None = None  # epoch ms dari browser (hanya untuk urutan; waktu server dipakai)
    value: float | None = None
    label: str | None = Field(default=None, max_length=256)


class CollectPayload(BaseModel):
    ctx: Ctx
    events: list[EventIn] = Field(max_length=50)


@router.post("/collect", status_code=204)
async def collect(request: Request, db: Session = Depends(get_db)):
    if _rate_limited(_client_ip(request), limit=600):  # longgar: satu IP bisa dipakai banyak orang (kampus/kantor)
        raise HTTPException(429, "Terlalu banyak permintaan")
    raw = await request.body()
    if len(raw) > 64_000:
        raise HTTPException(413, "Payload terlalu besar")
    try:
        payload = CollectPayload.model_validate_json(raw)
    except ValidationError:
        raise HTTPException(422, "Format data tidak valid")

    c = payload.ctx
    vid, sid = str(c.visitor_id), str(c.session_id)
    now = utcnow()
    path = normalize_path(c.page_url)
    slug = (c.program_slug or "").strip().lower()[:128] or program_from_path(path)
    own_host = urlparse(get_settings().public_api_url).hostname or ""
    own = tuple(h for h in ("ysii.org", own_host.split(".", 1)[-1]) if h)
    source, medium = classify_source(c.referrer, c.utm_source, c.utm_medium, own_hosts=own)

    visitor = db.get(Visitor, vid)
    if visitor is None:
        visitor = Visitor(
            visitor_id=vid, ref_code=ref_code_for(vid), first_seen_at=now, last_seen_at=now,
            first_source=source, first_medium=medium, first_campaign=c.utm_campaign,
            first_landing=path, device_type=c.device,
        )
        db.add(visitor)
    visitor.last_seen_at = now

    sess = db.get(VisitSession, sid)
    if sess is None:
        sess = VisitSession(
            session_id=sid, visitor_id=vid, started_at=now, ended_at=now, source=source,
            medium=medium, campaign=c.utm_campaign, utm_term=c.utm_term, landing_page=path,
            referrer=c.referrer, page_count=0, active_seconds=0, max_scroll_pct=0,
        )
        db.add(sess)
    elif sess.visitor_id != vid:
        raise HTTPException(422, "Sesi tidak cocok dengan pengunjung")
    sess.ended_at = now

    category = page_category(path, slug)
    for e in payload.events:
        if e.type not in ALLOWED_EVENTS:
            continue
        value = e.value
```

```
        if e.type == "page_view":
            sess.page_count += 1
        elif e.type == "heartbeat":
            value = max(0.0, min(float(value or 0), 1800.0))
            sess.active_seconds += int(value)
        elif e.type == "scroll":
            value = max(0.0, min(float(value or 0), 100.0))
            sess.max_scroll_pct = max(sess.max_scroll_pct, int(value))
        db.add(
            Event(
                session_id=sid, visitor_id=vid, ts=now, type=e.type, page_url=path,
                page_category=category, program_slug=slug, value=value, label=e.label,
            )
        )
    db.commit()
    return Response(status_code=204)


# ---------- /leads (form website) ----------
_PHONE_RE = re.compile(r"[^\d+]")
_PHONE_VALUE = re.compile(r"^\+?[\d\s().-]{8,20}$")
_EMAIL_VALUE = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")
# Potongan nama kolom yang dikenali (Contact Form 7, Elementor, WPForms, form HTML biasa)
_KNOWN_FIELDS = {
    "name": ("nama", "name", "fullname"),
    "phone": ("phone", "telepon", "telp", "no_hp", "nohp", "handphone", "whatsapp", "no_wa",
"hp", "wa", "tel"),
    "email": ("email", "e-mail", "surel"),
    "program_slug": ("program", "pelatihan", "kursus", "skema", "kelas", "training"),
    "message": ("message", "pesan", "catatan", "keterangan", "pertanyaan"),
}
_SKIP_EXTRA = {"visitor_id", "website", "_redirect", "_wpcf7", "_wpnonce", "page_url"}
_INTERNAL_PREFIX = ("_wpcf7", "_wpnonce", "action", "post_id", "form_id", "queried_id", "refer
er_title")


def normalize_phone(phone: str | None) -> str | None:
    if not phone:
        return None
    p = _PHONE_RE.sub("", phone)
    if p.startswith("+"):
        p = p[1:]
    if p.startswith("0"):
        p = "62" + p[1:]
    return p[:20] or None


def _field_key(key: str) -> str:
    """'form_fields[phone]' -> 'phone', 'your-name' -> 'your-name'."""
    inner = re.findall(r"\[([^\]]+)\]", key)
    return (inner[-1] if inner else key).strip().lower()


def _matches(key: str, names: tuple[str, ...]) -> bool:
    k = _field_key(key)
    tokens = set(re.split(r"[^a-z0-9]+", k))
    return any(n == k or n in tokens or (len(n) > 3 and n in k) for n in names)


# "nama" paling umum dipakai sebagai awalan (mis. "nama_program"), jadi dicek paling akhir
_FIELD_ORDER = ("email", "phone", "program_slug", "message", "name")


def _field_of(key: str) -> str | None:
    for field in _FIELD_ORDER:
        names = _KNOWN_FIELDS[field]
        if _matches(key, names):
            return field
```

```
    return None


def parse_lead_fields(data: dict) -> dict:
    """Petakan data form apa pun ke field lead; sisanya masuk 'extra'."""
    out: dict = {"extra": {}}
    for k, v in data.items():
        if v in (None, "") or k in _SKIP_EXTRA or k.startswith(_INTERNAL_PREFIX):
            continue
        field = _field_of(k)
        if field and field not in out:
            out[field] = str(v).strip()[:2000]
        else:
            out["extra"][k[:100]] = str(v)[:500]
    # Cadangan: kenali dari isi (mis. WPForms memakai nama kolom berupa angka)
    for k, v in list(out["extra"].items()):
        if "phone" not in out and _PHONE_VALUE.match(v) and sum(c.isdigit() for c in v) >= 8:
            out["phone"] = out["extra"].pop(k)
        elif "email" not in out and _EMAIL_VALUE.match(v):
            out["email"] = out["extra"].pop(k)
    return out


def _valid_uuid(value: str | None) -> str | None:
    try:
        return str(uuid.UUID(str(value))) if value else None
    except ValueError:
        return None


def notify_hot_lead(lead_id: int) -> None:
    url = get_settings().hot_lead_webhook_url
    if not url:
        return
    with SessionLocal() as db:
        lead = db.get(Lead, lead_id)
        if lead is None:
            return
        body = {
            "lead_id": lead.lead_id, "name": lead.name, "phone": lead.phone,
            "program": lead.program_slug, "score": lead.score, "segment": lead.segment,
            "text": f"Lead HOT baru: {lead.name or '-'} ({lead.phone or '-'}) "
                    f"program {lead.program_slug or '-'}, skor {round((lead.score or 0) * 10
0)}%",
        }
    try:
        httpx.post(url, json=body, timeout=10)
    except httpx.HTTPError as exc:
        log.warning("Gagal mengirim notifikasi lead hot: %s", exc)


def score_lead(db: Session, lead: Lead) -> dict | None:
    """Hitung skor lead dari perilaku pengunjung SEBELUM mendaftar."""
    if not lead.visitor_id:
        return None
    feats = visitor_features(db, lead.visitor_id, as_of=lead.created_at)
    if feats is None:
        return None
    result = score_features(db, feats, task="lead")
    lead.score, lead.segment = result["score"], result["segment"]
    db.add(
        Prediction(
            visitor_id=lead.visitor_id, lead_id=lead.lead_id, model_version=result["model_vers
ion"],
            score=result["score"], segment=result["segment"],
        )
    )
    return result
```

```


def create_lead(db: Session, data: dict, channel: str, visitor_id: str | None) -> Lead:
    if visitor_id and db.get(Visitor, visitor_id) is None:
        visitor_id = None
    f = parse_lead_fields(data)
    program = f.get("program_slug")
    program = re.sub(r"[^a-z0-9]+", "-", program.lower()).strip("-")[:128] if program else Non
e
    phone = normalize_phone(f.get("phone"))
    email = f.get("email")

    # Form yang terkirim ulang dalam 30 menit dianggap lead yang sama
    dup_filter = [Lead.phone == phone] if phone else [Lead.email == email]
    recent = db.scalar(
        select(Lead)
        .where(*dup_filter, Lead.created_at >= utcnow() - timedelta(minutes=30))
        .order_by(Lead.created_at.desc())
    )
    if recent is not None:
        recent.name = f.get("name") or recent.name
        recent.email = email or recent.email
        recent.program_slug = program or recent.program_slug
        recent.message = f.get("message") or recent.message
        recent.visitor_id = recent.visitor_id or visitor_id
        recent.updated_at = utcnow()
        if recent.score is None:
            score_lead(db, recent)
        db.commit()
        return recent

    lead = Lead(
        visitor_id=visitor_id,
        channel=channel,
        name=f.get("name"),
        phone=phone,
        email=email,
        program_slug=program,
        message=f.get("message"),
        extra=f["extra"] or None,
        created_at=utcnow(),
        updated_at=utcnow(),
    )
    db.add(lead)
    db.flush()
    score_lead(db, lead)
    db.commit()
    return lead


def _safe_redirect(url: str | None) -> str | None:
    if not url:
        return None
    host = urlparse(url).hostname or ""
    allowed = {urlparse(o).hostname for o in get_settings().origins}
    return url if host in allowed else None


@router.post("/leads")
async def submit_lead(
    request: Request, background: BackgroundTasks, db: Session = Depends(get_db)
):
    """Terima data form pendaftaran dari website (JSON atau form-urlencoded/multipart)."""
    if _rate_limited(_client_ip(request) + ":lead", limit=10):
        raise HTTPException(429, "Terlalu banyak permintaan")
    ctype = request.headers.get("content-type", "")
    if "application/json" in ctype or "text/plain" in ctype:
        try:
```

```
            data = json.loads(await request.body() or b"{}")
        except json.JSONDecodeError:
            raise HTTPException(422, "JSON tidak valid")
        if not isinstance(data, dict):
            raise HTTPException(422, "JSON harus berupa objek")
    else:
        form = await request.form()
        data = {k: v for k, v in form.items() if isinstance(v, str)}

    redirect = _safe_redirect(data.get("_redirect"))
    if data.get("website"):  # honeypot: diisi bot, bukan manusia
        return RedirectResponse(redirect, 303) if redirect else {"ok": True}
    fields = parse_lead_fields(data)
    if not (fields.get("phone") or fields.get("email")):
        raise HTTPException(422, "Nomor WhatsApp/telepon atau email wajib diisi")

    lead = create_lead(db, data, "form", _valid_uuid(data.get("visitor_id")))
    if lead.segment == "hot":
        background.add_task(notify_hot_lead, lead.lead_id)
    if redirect:
        return RedirectResponse(redirect, status_code=303)
    return {"ok": True, "lead_id": lead.lead_id}


# ---------- /recommend ----------
@router.get("/recommend/{visitor_id}")
def recommend(visitor_id: str, current: str | None = None, db: Session = Depends(get_db)):
    vid = _valid_uuid(visitor_id)
    segment = "cold"
    if vid:
        feats = visitor_features(db, vid)
        if feats is not None:
            segment = score_features(db, feats, task="visitor")["segment"]
    programs = recommend_programs(db, vid or "", current=(current or "").lower() or None)
    return {"segment": segment, "cta": CTA_BY_SEGMENT[segment], "programs": programs}


@router.get("/health", tags=["sistem"])
def health():
    return {"status": "ok", "time": datetime.now(timezone.utc).isoformat()}
```

backend/app/routers/admin.py — API dashboard admin (489 baris)

```
"""Endpoint dashboard manajemen lead (wajib login admin)."""

import csv
import io
import threading
from collections import Counter, defaultdict
from datetime import date, datetime, timedelta, timezone
from zoneinfo import ZoneInfo

from fastapi import APIRouter, Depends, HTTPException, Query, Request
from fastapi.responses import StreamingResponse
from pydantic import BaseModel, Field
from sqlalchemy import func, or_, select
from sqlalchemy.orm import Session

from ..config import get_settings
from ..db import SessionLocal, get_db
from ..features import visitor_features
from ..ml_engine import OPEN_STATUSES, TASKS, InsufficientDataError, train_all, train_models
from ..models import (
    CONVERTED_STATUSES,
    LEAD_STATUSES,
    AdminUser,
    Event,
    GscDaily,
```

```
    Lead,
    ModelRegistry,
    Program,
    Visitor,
    VisitSession,
    utcnow,
)
from ..scoring import RULE_VERSION, score_features
from ..security import create_token, current_admin, verify_password
from .public import _client_ip, _rate_limited, create_lead, score_lead

TZ = ZoneInfo("Asia/Jakarta")

auth_router = APIRouter(prefix="/v1/auth", tags=["auth"])
router = APIRouter(prefix="/v1/admin", tags=["admin"], dependencies=[Depends(current_admin)])


# ---------------- auth ----------------
class LoginIn(BaseModel):
    username: str
    password: str


@auth_router.post("/login")
def login(body: LoginIn, request: Request, db: Session = Depends(get_db)):
    if _rate_limited(_client_ip(request) + ":login", limit=10):
        raise HTTPException(429, "Terlalu banyak percobaan login, coba lagi 1 menit lagi")
    user = db.scalar(select(AdminUser).where(AdminUser.username == body.username))
    if user is None or not user.is_active or not verify_password(body.password, user.password_
hash):
        raise HTTPException(401, "Username atau password salah")
    return {"access_token": create_token(user.username), "token_type": "bearer"}


@router.get("/me")
def me(user: AdminUser = Depends(current_admin)):
    return {"username": user.username, "whatsapp_admin": get_settings().whatsapp_admin}


# ---------------- helpers ----------------
def _since(days: int) -> datetime:
    return datetime.now(timezone.utc) - timedelta(days=days)


def _local_day(dt: datetime) -> str:
    if dt.tzinfo is None:
        dt = dt.replace(tzinfo=timezone.utc)
    return dt.astimezone(TZ).date().isoformat()


def _lead_dict(lead: Lead, programs: dict[str, str] | None = None) -> dict:
    return {
        "lead_id": lead.lead_id,
        "created_at": lead.created_at.isoformat() if lead.created_at else None,
        "channel": lead.channel,
        "name": lead.name,
        "phone": lead.phone,
        "email": lead.email,
        "program_slug": lead.program_slug,
        "program_name": (programs or {}).get(lead.program_slug or "", None),
        "message": lead.message,
        "extra": lead.extra,
        "status": lead.status,
        "notes": lead.notes,
        "converted": lead.converted,
        "score": lead.score,
        "segment": lead.segment,
        "visitor_id": lead.visitor_id,
```

```
    }


def _program_names(db: Session) -> dict[str, str]:
    return {p.slug: p.name for p in db.scalars(select(Program))}


# ---------------- KPI ----------------
@router.get("/kpi")
def kpi(days: int = Query(30, ge=1, le=365), db: Session = Depends(get_db)):
    since = _since(days)
    visitors = db.scalars(select(Visitor).where(Visitor.last_seen_at >= since)).all()
    sessions = db.scalars(select(VisitSession).where(VisitSession.started_at >= since)).all()
    leads = db.scalars(select(Lead).where(Lead.created_at >= since)).all()

    n_visitors = len(visitors)
    n_leads = len(leads)
    n_converted = sum(1 for l in leads if l.converted)
    by_source = Counter(f"{s.source} / {s.medium}" for s in sessions)
    organic_sessions = sum(1 for s in sessions if s.medium == "organic")

    daily: dict[str, dict] = defaultdict(lambda: {"sessions": 0, "organic": 0, "leads": 0, "co
nverted": 0})
    for s in sessions:
        d = daily[_local_day(s.started_at)]
        d["sessions"] += 1
        d["organic"] += s.medium == "organic"
    for l in leads:
        d = daily[_local_day(l.created_at)]
        d["leads"] += 1
        d["converted"] += bool(l.converted)
    start = datetime.now(TZ).date() - timedelta(days=days - 1)
    series = []
    for i in range(days):
        key = (start + timedelta(days=i)).isoformat()
        series.append({"date": key, **daily.get(key, {"sessions": 0, "organic": 0, "leads": 0,
"converted": 0})})

    active = db.scalar(
        select(ModelRegistry).where(ModelRegistry.is_active.is_(True), ModelRegistry.task ==
"lead")
    )
    model_info = {"version": RULE_VERSION, "algorithm": "aturan (rule-based)", "metrics": Non
e}
    if active:
        best = active.metrics["comparison"][active.algorithm]
        model_info = {
            "version": active.version, "algorithm": active.algorithm, "task_label": TASKS["lea
d"],
            "metrics": best["cv"], "trained_at": active.trained_at.isoformat(),
        }

    return {
        "days": days,
        "visitors": n_visitors,
        "sessions": len(sessions),
        "organic_sessions": organic_sessions,
        "leads": n_leads,
        "converted": n_converted,
        "visitor_to_lead_rate": round(n_leads / n_visitors, 4) if n_visitors else 0,
        "lead_to_participant_rate": round(n_converted / n_leads, 4) if n_leads else 0,
        "segments": Counter(l.segment or "tanpa_skor" for l in leads),
        "statuses": Counter(l.status for l in leads),
        "channels": Counter(l.channel for l in leads),
        "sources": by_source.most_common(8),
        "series": series,
        "model": model_info,
    }
```

```
156
157
158 # ---------------- leads ----------------
159 @router.get("/leads")
160 def list_leads(
161     segment: str | None = None,
162     status: str | None = None,
163     program: str | None = None,
164     q: str | None = None,
165     page: int = Query(1, ge=1),
166     limit: int = Query(25, ge=1, le=200),
167     sort: str = Query("score", pattern="^(score|newest)$"),
168     db: Session = Depends(get_db),
169 ):
170     stmt = select(Lead)
171     if segment:
172         stmt = stmt.where(Lead.segment == segment)
173     if status == "berjalan":
174         stmt = stmt.where(Lead.status.in_(OPEN_STATUSES))
175     elif status:
176         stmt = stmt.where(Lead.status == status)
177     if program:
178         stmt = stmt.where(Lead.program_slug == program)
179     if q:
180         like = f"%{q}%"
181         stmt = stmt.where(or_(Lead.name.ilike(like), Lead.phone.ilike(like), Lead.email.ilike
    (like)))
182     total = db.scalar(select(func.count()).select_from(stmt.subquery()))
183     if sort == "score":
184         stmt = stmt.order_by(Lead.score.desc().nulls_last(), Lead.created_at.desc())
185     else:
186         stmt = stmt.order_by(Lead.created_at.desc())
187     rows = db.scalars(stmt.offset((page - 1) * limit).limit(limit)).all()
188     names = _program_names(db)
189     return {"total": total, "page": page, "limit": limit, "items": [_lead_dict(l, names) for l
    in rows]}
190
191
192 @router.get("/leads/{lead_id}")
193 def lead_detail(lead_id: int, db: Session = Depends(get_db)):
194     lead = db.get(Lead, lead_id)
195     if lead is None:
196         raise HTTPException(404, "Lead tidak ditemukan")
197     out = {"lead": _lead_dict(lead, _program_names(db)), "visitor": None, "sessions": [], "tim
    eline": [], "score": None}
198     if lead.visitor_id:
199         v = db.get(Visitor, lead.visitor_id)
200         out["visitor"] = {
201             "ref_code": v.ref_code, "first_seen_at": v.first_seen_at.isoformat(),
202             "first_source": v.first_source, "first_medium": v.first_medium,
203             "first_landing": v.first_landing, "device_type": v.device_type,
204         }
205         sess = db.scalars(
206             select(VisitSession).where(VisitSession.visitor_id == v.visitor_id).order_by(Visit
    Session.started_at)
207         ).all()
208         out["sessions"] = [
209             {
210                 "started_at": s.started_at.isoformat(), "source": s.source, "medium": s.mediu
    m,
211                 "landing_page": s.landing_page, "utm_term": s.utm_term, "page_count": s.page_c
    ount,
212                 "active_seconds": s.active_seconds, "max_scroll_pct": s.max_scroll_pct,
213             }
214             for s in sess
215         ]
216         evs = db.scalars(
217             select(Event)
```

```
            .where(Event.visitor_id == v.visitor_id, Event.type.not_in(["heartbeat", "scrol
l"]))
            .order_by(Event.ts)
            .limit(300)
        ).all()
        out["timeline"] = [
            {"ts": e.ts.isoformat(), "type": e.type, "page": e.page_url, "program": e.program_
slug, "label": e.label}
            for e in evs
        ]
        feats = visitor_features(db, v.visitor_id, as_of=lead.created_at)
        if feats:
            out["score"] = score_features(db, feats, task="lead") | {"features": {k: feats[k]
for k in sorted(feats)}}
    return out


class LeadUpdate(BaseModel):
    status: str | None = None
    notes: str | None = Field(default=None, max_length=5000)
    program_slug: str | None = None


@router.patch("/leads/{lead_id}")
def update_lead(lead_id: int, body: LeadUpdate, db: Session = Depends(get_db)):
    lead = db.get(Lead, lead_id)
    if lead is None:
        raise HTTPException(404, "Lead tidak ditemukan")
    if body.status is not None:
        if body.status not in LEAD_STATUSES:
            raise HTTPException(422, f"Status harus salah satu dari {LEAD_STATUSES}")
        lead.status = body.status
        # Label untuk retraining (closed-loop learning)
        if body.status in CONVERTED_STATUSES:
            if not lead.converted:
                lead.converted_at = utcnow()
            lead.converted = True
        elif body.status == "batal":
            lead.converted, lead.converted_at = False, None
        else:
            lead.converted, lead.converted_at = None, None
    if body.notes is not None:
        lead.notes = body.notes
    if body.program_slug is not None:
        lead.program_slug = body.program_slug or None
    lead.updated_at = utcnow()
    db.commit()
    return _lead_dict(lead, _program_names(db))


class ManualLeadIn(BaseModel):
    name: str | None = None
    phone: str
    email: str | None = None
    program_slug: str | None = None
    message: str | None = None
    ref_code: str | None = Field(default=None, description="Kode referensi dari pesan WhatsAp
p")


@router.post("/leads", status_code=201)
def create_manual_lead(body: ManualLeadIn, db: Session = Depends(get_db)):
    """Catat lead yang masuk lewat chat WhatsApp.

    Pesan WhatsApp dari website otomatis berisi 'Kode: XXXXXX'. Masukkan kode itu
    supaya lead terhubung dengan riwayat perilaku pengunjung di website.
    """
    visitor_id = None
```

```
    if body.ref_code:
        code = body.ref_code.strip().upper().lstrip("#")
        visitor_id = db.scalar(
            select(Visitor.visitor_id).where(Visitor.ref_code == code).order_by(Visitor.last_s
een_at.desc())
        )
        if visitor_id is None:
            raise HTTPException(404, f"Kode referensi {code} tidak ditemukan")
    lead = create_lead(db, body.model_dump(exclude={"ref_code"}), "whatsapp", visitor_id)
    return _lead_dict(lead, _program_names(db))


@router.post("/leads/{lead_id}/rescore")
def rescore(lead_id: int, db: Session = Depends(get_db)):
    lead = db.get(Lead, lead_id)
    if lead is None:
        raise HTTPException(404, "Lead tidak ditemukan")
    result = score_lead(db, lead)
    db.commit()
    return result or {"detail": "Lead tidak terhubung dengan data pengunjung"}


@router.get("/export/leads.csv")
def export_leads(db: Session = Depends(get_db)):
    names = _program_names(db)
    buf = io.StringIO()
    w = csv.writer(buf)
    cols = ["lead_id", "created_at", "channel", "name", "phone", "email", "program_slug",
            "program_name", "status", "converted", "score", "segment", "message", "notes"]
    w.writerow(cols)
    for lead in db.scalars(select(Lead).order_by(Lead.created_at.desc())):
        d = _lead_dict(lead, names)
        w.writerow([d[c] for c in cols])
    return StreamingResponse(
        iter([buf.getvalue()]), media_type="text/csv",
        headers={"Content-Disposition": "attachment; filename=leads.csv"},
    )


# ---------------- programs ----------------
class ProgramIn(BaseModel):
    slug: str = Field(pattern=r"^[a-z0-9-]{2,128}$")
    name: str
    category: str | None = None
    price: float | None = None
    url: str | None = None
    next_batch: date | None = None
    active: bool = True


@router.get("/programs")
def list_programs(db: Session = Depends(get_db)):
    return [ProgramIn.model_validate(p, from_attributes=True).model_dump(mode="json")
            for p in db.scalars(select(Program).order_by(Program.name))]


@router.put("/programs/{slug}")
def upsert_program(slug: str, body: ProgramIn, db: Session = Depends(get_db)):
    if slug != body.slug:
        raise HTTPException(422, "Slug di URL dan body berbeda")
    p = db.get(Program, slug) or Program(slug=slug)
    for k, v in body.model_dump().items():
        setattr(p, k, v)
    db.add(p)
    db.commit()
    return body.model_dump(mode="json")


```

```
@router.delete("/programs/{slug}", status_code=204)
def delete_program(slug: str, db: Session = Depends(get_db)):
    p = db.get(Program, slug)
    if p:
        db.delete(p)
        db.commit()


# ---------------- SEO ----------------
@router.get("/seo")
def seo(days: int = Query(28, ge=7, le=480), db: Session = Depends(get_db)):
    since = date.today() - timedelta(days=days)
    rows = db.scalars(select(GscDaily).where(GscDaily.date >= since)).all()
    if not rows:
        return {"days": days, "available": False}
    clicks = sum(r.clicks for r in rows)
    impressions = sum(r.impressions for r in rows)
    pos_weighted = sum(r.position * r.impressions for r in rows)

    def top(key):
        agg: dict[str, dict] = defaultdict(lambda: {"clicks": 0, "impressions": 0, "pos": 0.
    0})
        for r in rows:
            a = agg[getattr(r, key)]
            a["clicks"] += r.clicks
            a["impressions"] += r.impressions
            a["pos"] += r.position * r.impressions
        out = [
            {key: k, "clicks": a["clicks"], "impressions": a["impressions"],
             "ctr": round(a["clicks"] / a["impressions"], 4) if a["impressions"] else 0,
             "position": round(a["pos"] / a["impressions"], 1) if a["impressions"] else None}
            for k, a in agg.items()
        ]
        return sorted(out, key=lambda d: (d["clicks"], d["impressions"]), reverse=True)[:20]

    daily: dict[str, dict] = defaultdict(lambda: {"clicks": 0, "impressions": 0})
    for r in rows:
        daily[r.date.isoformat()]["clicks"] += r.clicks
        daily[r.date.isoformat()]["impressions"] += r.impressions
    pages = top("page")
    return {
        "days": days,
        "available": True,
        "clicks": clicks,
        "impressions": impressions,
        "ctr": round(clicks / impressions, 4) if impressions else 0,
        "avg_position": round(pos_weighted / impressions, 1) if impressions else None,
        "top_queries": top("query"),
        "top_pages": pages,
        "low_ctr_pages": [p for p in pages if p["impressions"] >= 100 and p["ctr"] < 0.02],
        "series": [{"date": d, **v} for d, v in sorted(daily.items())],
    }


# ---------------- model ----------------
_train_state: dict = {"running": False, "last_error": None, "last_result": None}


def _run_training(task: str, force: bool) -> None:
    _train_state.update(running=True, last_error=None, last_result=None)
    try:
        with SessionLocal() as db:
            if task == "all":
                results = train_all(db, force=force)
            else:
                try:
                    results = {task: train_models(db, task=task, force=force)}
                except InsufficientDataError as exc:
```

```
                    results = {task: {"error": str(exc)}}
        _train_state["last_result"] = {
            t: r.get("version") or r.get("error") for t, r in results.items()
        }
    except RuntimeError as exc:
        _train_state["last_error"] = str(exc)
    except Exception as exc:  # noqa: BLE001 — tampilkan ke admin, jangan matikan server
        _train_state["last_error"] = f"Gagal melatih model: {exc}"
    finally:
        _train_state["running"] = False


def _count(db: Session, *where) -> int:
    return db.scalar(select(func.count()).select_from(Lead).where(*where)) or 0


@router.get("/model")
def model_info(db: Session = Depends(get_db)):
    history = db.scalars(select(ModelRegistry).order_by(ModelRegistry.trained_at.desc()).limit(30)).all()
    return {
        "training": _train_state,
        "tasks": TASKS,
        "rule_version": RULE_VERSION,
        "min_samples": get_settings().min_positive_samples,
        "labeled": {
            "leads": _count(db),
            "linked": _count(db, Lead.visitor_id.is_not(None)),
            "converted": _count(db, Lead.converted.is_(True)),
            "batal": _count(db, Lead.converted.is_(False)),
        },
        "active": {
            t: next((m.version for m in history if m.task == t and m.is_active), None) for t in TASKS
        },
        "models": [
            {"version": m.version, "task": m.task, "algorithm": m.algorithm,
             "trained_at": m.trained_at.isoformat(), "n_samples": m.n_samples,
             "n_positive": m.n_positive, "is_active": m.is_active, "report": m.metrics}
            for m in history
        ],
    }


class RetrainIn(BaseModel):
    task: str = Field("all", pattern="^(all|lead|visitor)$")
    force: bool = False


@router.post("/model/retrain", status_code=202)
def retrain(body: RetrainIn):
    if _train_state["running"]:
        raise HTTPException(409, "Pelatihan sedang berjalan")
    _train_state["running"] = True
    threading.Thread(target=_run_training, args=(body.task, body.force), daemon=True).start()
    return {"started": True}


@router.post("/model/{version}/activate")
def activate_model(version: str, db: Session = Depends(get_db)):
    target = db.get(ModelRegistry, version)
    if target is None:
        raise HTTPException(404, "Versi model tidak ditemukan")
    for m in db.scalars(select(ModelRegistry).where(ModelRegistry.task == target.task)):
        m.is_active = m.version == version
    db.commit()
    return {"task": target.task, "active": version}

```

```
482
483 @router.post("/model/deactivate")
484 def deactivate_models(task: str = Query(..., pattern="^(lead|visitor)$"), db: Session = Depend
    s(get_db)):
485     """Kembali ke skor berbasis aturan untuk satu tugas."""
486     for m in db.scalars(select(ModelRegistry).where(ModelRegistry.task == task)):
487         m.is_active = False
488     db.commit()
489     return {"task": task, "active": RULE_VERSION}
```

backend/ml/train.py — Perintah pelatihan model (67 baris)

```
  1 """Latih & bandingkan Logistic Regression, Random Forest, Gradient Boosting.
  2
  3 Tugas:
  4   lead    = lead -> menjadi peserta (prioritas lead di dashboard)
  5   visitor = pengunjung -> menjadi lead (rekomendasi & CTA di website)
  6
  7     python -m ml.train                          # semua tugas yang datanya cukup
  8     python -m ml.train --task lead --force     # paksa walau data sedikit
  9     python -m ml.train --task lead --export-dataset lead.csv   # dataset untuk notebook
 10 """
 11
 12 import argparse
 13 import json
 14
 15 from app.config import get_settings
 16 from app.db import SessionLocal, init_db
 17 from app.ml_engine import TASKS, InsufficientDataError, build_dataset, train_models
 18
 19
 20 def main() -> None:
 21     p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHe
    lpFormatter)
 22     p.add_argument("--task", choices=["all", *TASKS], default="all")
 23     p.add_argument("--force", action="store_true", help="Latih walau contoh positif di bawah b
    atas minimal")
 24     p.add_argument("--no-activate", action="store_true", help="Simpan model tanpa mengaktifkan
    nya")
 25     p.add_argument("--export-dataset", metavar="CSV", help="Hanya ekspor dataset fitur ke CS
    V")
 26     args = p.parse_args()
 27
 28     init_db()
 29     tasks = list(TASKS) if args.task == "all" else [args.task]
 30     with SessionLocal() as db:
 31         if args.export_dataset:
 32             for task in tasks:
 33                 df = build_dataset(db, task)
 34                 path = args.export_dataset if len(tasks) == 1 else f"{task}-{args.export_datas
    et}"
 35                 df.to_csv(path, index=False)
 36                 print(f"[{task}] {len(df)} baris ({int(df['y'].sum()) if len(df) else 0} posit
    if) -> {path}")
 37             return
 38         for task in tasks:
 39             try:
 40                 report = train_models(db, task=task, force=args.force, activate=not args.no_ac
    tivate)
 41             except InsufficientDataError as exc:
 42                 print(f"\n{exc}")
 43                 continue
 44             _print(report, activated=not args.no_activate)
 45
 46
 47 def _print(report: dict, activated: bool) -> None:
 48     print(f"\n=== {report['task_label']} ===")
```

```
    print(f"Data: {report['n_samples']} ({report['n_positive']} positif) | {report['cv_folds']}-fold stratified CV\n")
    print(f"{'Algoritma':<22}{'Akurasi':>9}{'Presisi':>9}{'Recall':>9}{'F1':>9}{'AUC':>9}{'F1 uji':>9}{'AUC uji':>9}")
    for name, r in report["comparison"].items():
        cv, ho = r["cv"], r["holdout"]
        star = " *" if name == report["best_algorithm"] else ""
        print(f"{name + star:<22}{cv['accuracy']:>9}{cv['precision']:>9}{cv['recall']:>9}{cv['f1']:>9}"
              f"{cv['roc_auc']:>9}{ho['f1']:>9}{ho.get('roc_auc', '-'):>9}")
    print("\nFitur terpenting:")
    for f in report["feature_importance"][:8]:
        print(f"  {f['feature']:<28}{f['importance']}")
    print(f"\nModel disimpan{'' if activated else ' (tidak diaktifkan)'}: {report['version']}")
    if "rescored_leads" in report:
        print(f"Skor {report['rescored_leads']} lead yang masih berjalan diperbarui.")
    with open(get_settings().model_dir / f"{report['version']}.report.json", "w") as fh:
        json.dump(report, fh, indent=2)


if __name__ == "__main__":
    main()
```

backend/jobs/gsc_etl.py — ETL Google Search Console (78 baris)

```
"""Tarik data Google Search Console (klik, impresi, CTR, posisi per halaman & kata kunci).

Persiapan:
1. Buat service account di Google Cloud, aktifkan "Google Search Console API", unduh kunci JSON.
2. Di Search Console > Setelan > Pengguna, tambahkan email service account (izin: Terbatas).
3. Set GSC_CREDENTIALS_FILE=/path/kunci.json dan GSC_SITE_URL=sc-domain:ysii.org

    python -m jobs.gsc_etl --days 90     # isi data awal (baseline)
    python -m jobs.gsc_etl               # harian (3 hari terakhir)
"""

import argparse
import logging
from datetime import date, timedelta

from sqlalchemy import delete

from app.config import get_settings
from app.db import SessionLocal, init_db
from app.models import GscDaily

log = logging.getLogger(__name__)


def fetch_day(service, site: str, day: date) -> list[dict]:
    rows, start = [], 0
    while True:
        resp = service.searchanalytics().query(
            siteUrl=site,
            body={
                "startDate": day.isoformat(), "endDate": day.isoformat(),
                "dimensions": ["page", "query"], "rowLimit": 25000, "startRow": start,
            },
        ).execute()
        batch = resp.get("rows", [])
        rows += batch
        if len(batch) < 25000:
            return rows
        start += 25000


```

```
def run(days: int = 3) -> int:
    s = get_settings()
    if not s.gsc_credentials_file:
        log.warning("GSC_CREDENTIALS_FILE belum diatur, ETL GSC dilewati")
        return 0
    from google.oauth2 import service_account
    from googleapiclient.discovery import build

    creds = service_account.Credentials.from_service_account_file(
        s.gsc_credentials_file, scopes=["https://www.googleapis.com/auth/webmasters.readonly"]
    )
    service = build("searchconsole", "v1", credentials=creds, cache_discovery=False)
    init_db()
    total = 0
    # data GSC baru final sekitar 2-3 hari kemudian
    for offset in range(days + 2, 2, -1):
        day = date.today() - timedelta(days=offset)
        rows = fetch_day(service, s.gsc_site_url, day)
        with SessionLocal() as db:
            db.execute(delete(GscDaily).where(GscDaily.date == day))
            for r in rows:
                page, query = r["keys"]
                db.add(GscDaily(
                    date=day, page=page[:512], query=query[:512], clicks=int(r["clicks"]),
                    impressions=int(r["impressions"]), ctr=float(r["ctr"]), position=float(r
["position"]),
                ))
            db.commit()
        log.info("GSC %s: %d baris", day, len(rows))
        total += len(rows)
    return total


if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)
    ap = argparse.ArgumentParser()
    ap.add_argument("--days", type=int, default=3)
    print(f"{run(ap.parse_args().days)} baris disimpan")
```

backend/jobs/scheduler.py — Penjadwal tugas rutin (37 baris)

```
"""Penjadwal tugas rutin:
- 06:00 WIB setiap hari : tarik data Google Search Console
- 02:00 WIB setiap Senin: latih ulang model (closed-loop learning)
"""

import logging

from apscheduler.schedulers.blocking import BlockingScheduler

from app.db import SessionLocal, init_db
from app.ml_engine import train_all
from jobs import gsc_etl

log = logging.getLogger("scheduler")


def weekly_retrain() -> None:
    with SessionLocal() as db:
        try:
            for task, result in train_all(db).items():
                log.info("Retrain %s: %s", task, result.get("version") or result.get("error"))
        except RuntimeError as exc:
            log.info("Retrain dilewati: %s", exc)


def main() -> None:
```

```
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(name)s: %(message)s")
    init_db()
    sched = BlockingScheduler(timezone="Asia/Jakarta")
    sched.add_job(gsc_etl.run, "cron", hour=6, minute=0, id="gsc_etl")
    sched.add_job(weekly_retrain, "cron", day_of_week="mon", hour=2, minute=0, id="retrain")
    log.info("Scheduler berjalan")
    sched.start()


if __name__ == "__main__":
    main()
```

backend/scripts/create_admin.py — Pembuatan akun admin (37 baris)

```
"""Buat / reset akun admin dashboard.

    python -m scripts.create_admin admin
"""

import argparse
import getpass

from sqlalchemy import select

from app.db import SessionLocal, init_db
from app.models import AdminUser
from app.security import hash_password


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("username")
    parser.add_argument("--password", help="Jika kosong, akan ditanyakan")
    args = parser.parse_args()
    password = args.password or getpass.getpass("Password baru: ")
    if len(password) < 10:
        raise SystemExit("Password minimal 10 karakter")
    init_db()
    with SessionLocal() as db:
        user = db.scalar(select(AdminUser).where(AdminUser.username == args.username))
        if user is None:
            user = AdminUser(username=args.username, password_hash="")
            db.add(user)
        user.password_hash = hash_password(password)
        user.is_active = True
        db.commit()
    print(f"Admin '{args.username}' siap dipakai.")


if __name__ == "__main__":
    main()
```

backend/app/static/tracker.js — Script pelacak di website (311 baris)

```
/*!
 * YSII Tracker — Sistem Promosi Digital Terintegrasi SEO & Machine Learning
 *
 * Pasang di <head> website:
 *   <script src="https://api.ysii.org/static/tracker.js" defer
 *           data-privacy-url="https://ysii.org/kebijakan-privasi"></script>
 *
 * Atribut opsional pada tag <script>:
 *   data-api            URL API (default: asal script ini)
 *   data-consent        "required" (default, tampilkan banner persetujuan) | "implied"
 *   data-privacy-url    tautan kebijakan privasi di banner
 *   data-mirror-forms   "auto" (default: salin form yang punya kolom telepon/email ke sistem lead)
```

```
 *                       | "marked" (hanya form ber-atribut data-ysii-form) | "off"
 *   data-whatsapp-ref   "on" (default: tambahkan kode referensi ke pesan WhatsApp) | "off"
 */
(function () {
  "use strict";
  if (window.ysii && window.ysii.loaded) return;

  var script = document.currentScript || document.querySelector('script[src*="tracker.js"]');
  var cfg = (script && script.dataset) || {};
  var API = (cfg.api || (script ? new URL(script.src).origin : "")).replace(/\/$/, "");
  var CONSENT_MODE = cfg.consent || "required";
  var MIRROR = cfg.mirrorForms || "auto";
  var WA_REF = cfg.whatsappRef !== "off";
  var SESSION_TIMEOUT = 30 * 60 * 1000;

  // ---------- util ----------
  function store(key, val) {
    try {
      if (val === undefined) return localStorage.getItem(key);
      if (val === null) localStorage.removeItem(key);
      else localStorage.setItem(key, val);
    } catch (e) { return null; }
  }
  function uuid() {
    if (window.crypto && crypto.randomUUID) return crypto.randomUUID();
    var b = new Uint8Array(16);
    (window.crypto || window.msCrypto).getRandomValues(b);
    b[6] = (b[6] & 0x0f) | 0x40; b[8] = (b[8] & 0x3f) | 0x80;
    var h = Array.prototype.map.call(b, function (x) { return (x + 0x100).toString(16).slice
(1); }).join("");
    return h.slice(0, 8) + "-" + h.slice(8, 12) + "-" + h.slice(12, 16) + "-" + h.slice(16, 2
0) + "-" + h.slice(20);
  }
  function send(path, data) {
    var body = JSON.stringify(data);
    var url = API + path;
    // text/plain = "simple request", tidak perlu preflight CORS
    if (navigator.sendBeacon && navigator.sendBeacon(url, new Blob([body], { type: "text/plai
n" }))) return;
    try { fetch(url, { method: "POST", body: body, keepalive: true, headers: { "Content-Type":
"text/plain" } }); } catch (e) {}
  }

  // ---------- program slug ----------
  function detectProgram() {
    var meta = document.querySelector('meta[name="ysii:program"]');
    if (meta && meta.content) return meta.content;
    if (document.body && document.body.dataset.program) return document.body.dataset.program;
    var m = location.pathname.match(/\/(?:program|pelatihan|kursus|sertifikasi|training|kelas)
\/([a-z0-9-]+)/i);
    return m ? m[1].toLowerCase() : null;
  }
  var PROGRAM = detectProgram();

  // ---------- identitas ----------
  var consent = store("ysii_consent"); // "1" setuju, "0" menolak, null belum memilih
  if (CONSENT_MODE === "implied" && consent === null) consent = "1";
  var tracking = consent === "1";
  var vid = null, sid = null, refCode = null;

  function initIds() {
    vid = store("ysii_vid");
    if (!vid) { vid = uuid(); store("ysii_vid", vid); }
    var now = Date.now(), last = +(store("ysii_last") || 0);
    sid = store("ysii_sid");
    var q = new URLSearchParams(location.search);
    var newCampaign = q.get("utm_source") && q.get("utm_source") !== store("ysii_utm");
```

```
    if (!sid || now - last > SESSION_TIMEOUT || newCampaign) { sid = uuid(); store("ysii_sid", sid); }
    if (q.get("utm_source")) store("ysii_utm", q.get("utm_source"));
    store("ysii_last", String(now));
    refCode = vid.replace(/-/g, "").slice(0, 6).toUpperCase();
  }

  // ---------- antrian event ----------
  var buf = [], ctx = null;
  function track(type, label, value) {
    if (!tracking) return;
    buf.push({ type: type, ts: Date.now(), label: label ? String(label).slice(0, 250) : null, value: value == null ? null : value });
    if (buf.length >= 20) flush();
  }
  function flush() {
    if (!tracking || !buf.length) return;
    store("ysii_last", String(Date.now()));
    send("/v1/collect", { ctx: ctx, events: buf.splice(0, 50) });
  }

  // ---------- waktu aktif ----------
  var activeSec = 0, lastTick = Date.now(), lastInput = Date.now();
  ["mousemove", "keydown", "scroll", "touchstart"].forEach(function (ev) {
    addEventListener(ev, function () { lastInput = Date.now(); }, { passive: true });
  });
  function tick() {
    var now = Date.now();
    // dihitung aktif hanya jika tab terlihat dan ada interaksi dalam 60 detik terakhir
    if (!document.hidden && now - lastInput < 60000) activeSec += (now - lastTick) / 1000;
    lastTick = now;
  }
  function heartbeat() {
    tick();
    if (activeSec >= 1) { track("heartbeat", null, Math.round(activeSec)); activeSec = 0; }
  }

  // ---------- WhatsApp ----------
  function isWa(href) { return /(^|\/\/)(wa\.me|api\.whatsapp\.com|web\.whatsapp\.com)|^whatsapp:/i.test(href || ""); }
  function addRef(a) {
    if (!WA_REF || !refCode || a.dataset.ysiiRef) return;
    try {
      var u = new URL(a.href, location.href);
      var text = u.searchParams.get("text") || "Halo admin YSII, saya ingin bertanya tentang pelatihan.";
      u.searchParams.set("text", text + "\n\n(Kode: " + refCode + ")");
      // URLSearchParams menulis spasi sebagai "+", yang tidak selalu dibaca sebagai spasi oleh WhatsApp
      a.href = u.toString().replace(/\+/g, "%20");
      a.dataset.ysiiRef = "1";
    } catch (e) {}
  }

  // ---------- form ----------
  var EXCLUDED_FORM = 'form[role="search"], #commentform, .search-form, form[action*="wp-login"], form[action*="/search"]';
  function shouldMirror(f) {
    if (MIRROR === "off" || f.matches(EXCLUDED_FORM) || f.querySelector('input[type="password"]')) return false;
    if (f.hasAttribute("data-ysii-form")) return true;
    if (MIRROR !== "auto") return false;
    return !!f.querySelector('input[type="tel"], input[type="email"], input[name*="phone" i], input[name*="hp" i], input[name*="whatsapp" i], input[name*="telepon" i]');
  }
  function formData(f) {
    var data = {};
    Array.prototype.forEach.call(f.elements, function (el) {
```

```
      if (!el.name || el.disabled || el.type === "password" || el.type === "file") return;
      if ((el.type === "checkbox" || el.type === "radio") && !el.checked) return;
      data[el.name] = data[el.name] ? data[el.name] + ", " + el.value : el.value;
    });
    return data;
  }
  function bindForm(f) {
    if (f.dataset.ysiiBound) return;
    f.dataset.ysiiBound = "1";
    if (tracking && vid && !f.querySelector('input[name="visitor_id"]')) {
      var h = document.createElement("input");
      h.type = "hidden"; h.name = "visitor_id"; h.value = vid;
      f.appendChild(h);
    }
    f.addEventListener("focusin", function once() {
      track("form_start", f.id || f.getAttribute("name") || null);
      f.removeEventListener("focusin", once);
    });
    f.addEventListener("submit", function () {
      track("form_submit", f.id || null);
      flush();
      if (shouldMirror(f)) {
        var data = formData(f);
        if (tracking && vid) data.visitor_id = vid; else delete data.visitor_id;
        if (PROGRAM && !data.program) data.program = PROGRAM;
        data.page_url = location.href;
        send("/v1/leads", data);
      }
    });
  }

  // ---------- klik ----------
  document.addEventListener("click", function (e) {
    var el = e.target.closest && e.target.closest("a, button, [data-track]");
    if (!el) return;
    var href = el.getAttribute("href") || "";
    var label = (el.dataset.trackLabel || el.textContent || "").trim().replace(/\s+/g, " ").sl
ice(0, 80);
    if (isWa(href)) { addRef(el); track("click_whatsapp", label || href); }
    else if (el.dataset.track) track(el.dataset.track, label);
    else if (/\.pdf($|\?)/i.test(href) || /brosur|brochure/i.test(href + label)) track("downlo
ad_brochure", label || href);
    else if (/daftar|registrasi|register|enroll/i.test(label) || el.matches(".cta, .btn-dafta
r")) track("click_cta", label);
  }, true);

  // ---------- scroll & bagian harga ----------
  var marks = {};
  function onScroll() {
    var h = document.documentElement.scrollHeight - innerHeight;
    var pct = h > 0 ? Math.round((scrollY / h) * 100) : 100;
    [25, 50, 75, 100].forEach(function (m) { if (pct >= m && !marks[m]) { marks[m] = 1; track
("scroll", null, m); } });
  }
  function watchPrice() {
    if (!("IntersectionObserver" in window)) return;
    var els = document.querySelectorAll('[data-track="view_price"], [data-ysii-price], #harga,
#biaya, #jadwal, .harga, .biaya');
    if (!els.length) return;
    var seen = false;
    var io = new IntersectionObserver(function (entries) {
      entries.forEach(function (en) {
        if (en.isIntersecting && !seen) { seen = true; track("view_price", en.target.id || nul
l); io.disconnect(); }
      });
    }, { threshold: 0.5 });
    Array.prototype.forEach.call(els, function (el) { io.observe(el); });
  }
```

```
197
198   // ---------- rekomendasi ----------
199   function esc(s) { return String(s == null ? "" : s).replace(/[&<>"']/g, function (c) { retur
    n { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c]; }); }
200   function safeUrl(u) { return /^(https?:|\/)/i.test(u || "") ? u : "#"; }
201   function renderReco() {
202     var box = document.getElementById("ysii-reco");
203     if (!box || !tracking) return;
204     var url = API + "/v1/recommend/" + vid + (PROGRAM ? "?current=" + encodeURIComponent(PROGR
    AM) : "");
205     fetch(url).then(function (r) { return r.json(); }).then(function (d) {
206       var wa = box.dataset.whatsapp, daftar = box.dataset.daftarUrl || "#";
207       var ctaHref = d.cta.type === "daftar" ? daftar : d.cta.type === "konsultasi" && wa
208         ? "https://wa.me/" + wa + "?text=" + encodeURIComponent("Halo admin YSII, saya ingin k
    onsultasi program pelatihan.") : (d.programs[0] && d.programs[0].url) || daftar;
209       var cards = d.programs.map(function (p) {
210         return '<a class="ysii-card" data-track="click_cta" data-track-label="reco:' + esc(p.s
    lug) + '" href="' + esc(safeUrl(p.url)) + '">' +
211           "<strong>" + esc(p.name) + "</strong>" + (p.next_batch ? "<span>Batch: " + esc(p.nex
    t_batch) + "</span>" : "") + "</a>";
212       }).join("");
213       box.innerHTML =
214         '<div class="ysii-reco"><h3>' + esc(d.cta.title) + "</h3><p>" + esc(d.cta.text) + "</p
    >" +
215         (cards ? '<div class="ysii-cards">' + cards + "</div>" : "") +
216         '<a class="ysii-btn" data-track="click_cta" data-track-label="reco-cta:' + esc(d.cta.t
    ype) + '" href="' + esc(safeUrl(ctaHref)) + '">' + esc(d.cta.button) + "</a></div>";
217       Array.prototype.forEach.call(box.querySelectorAll("a"), function (a) { if (isWa(a.href))
    addRef(a); });
218     }).catch(function () {});
219   }
220   function injectStyle() {
221     var css =
222       ".ysii-reco{border:1px solid #e3e6ea;border-radius:12px;padding:16px;margin:16px 0;font-
    family:inherit}" +
223       ".ysii-reco h3{margin:0 0 6px;font-size:1.1em}.ysii-reco p{margin:0 0 12px;color:#555}"
    +
224       ".ysii-cards{display:grid;grid-template-columns:repeat(auto-fit,minmax(180px,1fr));gap:1
    0px;margin-bottom:12px}" +
225       ".ysii-card{display:flex;flex-direction:column;gap:4px;padding:10px 12px;border-radius:8
    px;background:#f5f7fa;color:inherit;text-decoration:none}" +
226       ".ysii-card span{font-size:.85em;color:#666}" +
227       ".ysii-btn{display:inline-block;padding:10px 16px;border-radius:8px;background:#0b7a4b;c
    olor:#fff!important;text-decoration:none;font-weight:600}" +
228       "#ysii-consent{position:fixed;left:16px;right:16px;bottom:16px;z-index:99999;max-width:5
    60px;margin:auto;background:#1f2933;color:#fff;" +
229       "padding:14px 16px;border-radius:10px;font:14px/1.5 system-ui,sans-serif;box-shadow:0 6p
    x 24px rgba(0,0,0,.25)}" +
230       "#ysii-consent a{color:#9fe3c1}#ysii-consent .b{margin-top:10px;display:flex;gap:8px;jus
    tify-content:flex-end}" +
231       "#ysii-consent button{border:0;border-radius:6px;padding:8px 14px;cursor:pointer;font:in
    herit}" +
232       "#ysii-consent .y{background:#0b7a4b;color:#fff}#ysii-consent .n{background:transparent;
    color:#ddd;border:1px solid #555}";
233     var s = document.createElement("style");
234     s.textContent = css;
235     document.head.appendChild(s);
236   }
237
238   // ---------- banner persetujuan (UU PDP No. 27/2022) ----------
239   function showConsent() {
240     var d = document.createElement("div");
241     d.id = "ysii-consent";
242     d.setAttribute("role", "dialog");
243     d.setAttribute("aria-label", "Persetujuan cookie");
244     d.innerHTML =
245       "Kami memakai cookie untuk memahami cara pengunjung menggunakan situs ini dan memberi re
    komendasi program yang sesuai." +
```

```
      (cfg.privacyUrl ? ' <a href="' + esc(safeUrl(cfg.privacyUrl)) + '">Kebijakan privasi</a>.' : "") +
      '<div class="b"><button class="n" type="button">Tolak</button><button class="y" type="button">Setuju</button></div>';
    document.body.appendChild(d);
    d.querySelector(".y").onclick = function () { store("ysii_consent", "1"); d.remove(); start(); };
    d.querySelector(".n").onclick = function () { store("ysii_consent", "0"); d.remove(); };
  }

  // ---------- mulai ----------
  var started = false;
  function start() {
    if (started) return;
    started = true;
    tracking = true;
    initIds();
    var q = new URLSearchParams(location.search);
    ctx = {
      visitor_id: vid, session_id: sid, page_url: location.href, page_title: document.title.slice(0, 290),
      referrer: document.referrer || null,
      utm_source: q.get("utm_source"), utm_medium: q.get("utm_medium"),
      utm_campaign: q.get("utm_campaign"), utm_term: q.get("utm_term"),
      program_slug: PROGRAM, device: /Mobi|Android|iPhone/i.test(navigator.userAgent) ? "mobile" : "desktop"
    };
    track("page_view", document.title);
    var search = q.get("s") || q.get("q") || q.get("search");
    if (search) track("site_search", search);
    Array.prototype.forEach.call(document.querySelectorAll("form"), bindForm);
    Array.prototype.forEach.call(document.querySelectorAll("a[href]"), function (a) { if (isWa(a.href)) addRef(a); });
    addEventListener("scroll", onScroll, { passive: true });
    onScroll();
    watchPrice();
    renderReco();
    setInterval(function () { heartbeat(); flush(); }, 15000);
    document.addEventListener("visibilitychange", function () {
      if (document.hidden) { heartbeat(); flush(); } else { lastTick = Date.now(); }
    });
    addEventListener("pagehide", function () { heartbeat(); flush(); });
    // form yang dimuat belakangan (popup / AJAX)
    if ("MutationObserver" in window) {
      new MutationObserver(function () {
        Array.prototype.forEach.call(document.querySelectorAll("form:not([data-ysii-bound])"), bindForm);
      }).observe(document.body, { childList: true, subtree: true });
    }
  }

  function boot() {
    injectStyle();
    if (tracking) start();
    else {
      // Tanpa persetujuan: tetap salin form pendaftaran (tanpa visitor_id), tanpa tracking
      Array.prototype.forEach.call(document.querySelectorAll("form"), bindForm);
      if (consent === null && CONSENT_MODE === "required") showConsent();
    }
  }

  window.ysii = {
    loaded: true,
    track: function (type, label, value) { track(type, label, value); },
    flush: flush,
    get visitorId() { return vid; },
    get refCode() { return refCode; },
    resetConsent: function () { store("ysii_consent", null); location.reload(); }
```

```
  };

  if (document.readyState === "loading") document.addEventListener("DOMContentLoaded", boot);
  else boot();
})();
```

backend/app/static/dashboard/index.html — Struktur halaman dashboard (148 baris)

```
<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="robots" content="noindex, nofollow">
  <title>Dashboard Lead YSII</title>
  <link rel="stylesheet" href="style.css">
  <script src="vendor/chart.umd.js" defer></script>
  <script src="app.js" defer></script>
</head>
<body>
  <section id="login-view" class="login" hidden>
    <form id="login-form" class="card login-card">
      <h1>Dashboard Lead YSII</h1>
      <p class="muted">Sistem Promosi Digital Terintegrasi SEO &amp; Machine Learning</p>
      <label>Username <input name="username" autocomplete="username" required></label>
      <label>Password <input name="password" type="password" autocomplete="current-password" required></label>
      <button class="btn primary" type="submit">Masuk</button>
      <p id="login-error" class="error" role="alert"></p>
    </form>
  </section>

  <div id="app-view" hidden>
    <header class="topbar">
      <strong class="brand">YSII Lead</strong>
      <nav id="tabs">
        <a href="#ringkasan">Ringkasan</a>
        <a href="#lead">Lead</a>
        <a href="#program">Program</a>
        <a href="#seo">SEO</a>
        <a href="#model">Model ML</a>
      </nav>
      <button id="logout" class="btn ghost" type="button">Keluar</button>
    </header>

    <main>
      <!-- RINGKASAN -->
      <section data-page="ringkasan" hidden>
        <div class="row between">
          <h2>Ringkasan</h2>
          <select id="kpi-days" aria-label="Periode">
            <option value="7">7 hari</option>
            <option value="30" selected>30 hari</option>
            <option value="90">90 hari</option>
          </select>
        </div>
        <div id="kpi-tiles" class="tiles"></div>
        <div class="grid2">
          <div class="card"><h3>Kunjungan &amp; lead harian</h3><canvas id="chart-daily" height="140"></canvas></div>
          <div class="card"><h3>Segmen lead</h3><canvas id="chart-segments" height="140"></canvas></div>
        </div>
        <div class="grid2">
          <div class="card"><h3>Sumber traffic teratas</h3><table class="table" id="sources-table"></table></div>
          <div class="card"><h3>Model prediksi aktif</h3><div id="model-summary"></div></div>
        </div>
```

```
      </section>

      <!-- LEAD -->
      <section data-page="lead" hidden>
        <div class="row between">
          <h2>Lead</h2>
          <div class="row">
            <a class="btn ghost" id="export-csv" href="#">Unduh CSV</a>
            <button class="btn primary" id="btn-add-wa" type="button">+ Lead dari WhatsApp</bu
tton>
          </div>
        </div>
        <form id="lead-filter" class="row filters">
          <input name="q" placeholder="Cari nama / HP / email">
          <select name="segment">
            <option value="">Semua segmen</option>
            <option value="hot">Hot</option><option value="warm">Warm</option><option value="c
old">Cold</option>
          </select>
          <select name="status"><option value="">Semua status</option></select>
          <select name="program"><option value="">Semua program</option></select>
          <select name="sort"><option value="score">Urut: skor tertinggi</option><option value
="newest">Urut: terbaru</option></select>
        </form>
        <div class="card"><table class="table" id="lead-table"></table></div>
        <div class="row between"><span id="lead-count" class="muted"></span>
          <div class="row"><button class="btn ghost" id="prev" type="button">‹ Sebelumnya</but
ton><button class="btn ghost" id="next" type="button">Berikutnya ›</button></div>
        </div>
      </section>

      <!-- PROGRAM -->
      <section data-page="program" hidden>
        <h2>Program Pelatihan</h2>
        <p class="muted">Slug harus sama dengan bagian URL halaman program di website (mis. <c
ode>/program/<b>digital-marketing</b></code>) atau atribut <code>data-program</code>.</p>
        <div class="card"><table class="table" id="program-table"></table></div>
        <form id="program-form" class="card form-grid">
          <h3>Tambah / ubah program</h3>
          <label>Slug <input name="slug" required pattern="[a-z0-9-]{2,128}" placeholder="digi
tal-marketing"></label>
          <label>Nama <input name="name" required></label>
          <label>Kategori <input name="category"></label>
          <label>Harga (Rp) <input name="price" type="number" min="0"></label>
          <label>URL halaman <input name="url" type="url" placeholder="https://ysii.org/progra
m/..."></label>
          <label>Batch berikutnya <input name="next_batch" type="date"></label>
          <label class="check"><input name="active" type="checkbox" checked> Aktif</label>
          <button class="btn primary" type="submit">Simpan</button>
        </form>
      </section>

      <!-- SEO -->
      <section data-page="seo" hidden>
        <h2>Performa SEO (Google Search Console)</h2>
        <div id="seo-body"></div>
      </section>

      <!-- MODEL -->
      <section data-page="model" hidden>
        <div class="row between">
          <h2>Model Machine Learning</h2>
          <div class="row">
            <select id="retrain-task" aria-label="Tugas model">
              <option value="all">Semua model</option>
              <option value="lead">Model lead → peserta</option>
              <option value="visitor">Model pengunjung → lead</option>
            </select>
```

```
            <label class="check"><input type="checkbox" id="retrain-force"> Paksa (data sedikit)</label>
            <button class="btn primary" id="btn-retrain" type="button">Latih ulang model</button>
          </div>
        </div>
        <div id="model-body"></div>
      </section>
    </main>
  </div>

  <!-- Detail lead -->
  <dialog id="lead-dialog" class="dialog wide"><div id="lead-detail"></div></dialog>

  <!-- Lead WhatsApp -->
  <dialog id="wa-dialog" class="dialog">
    <form id="wa-form" method="dialog" class="form-grid">
      <h3>Catat lead dari WhatsApp</h3>
      <p class="muted">Pesan WhatsApp dari website berisi <b>(Kode: XXXXXX)</b>. Isi kode itu agar riwayat kunjungan calon peserta ikut terhubung.</p>
      <label>Nama <input name="name"></label>
      <label>No. WhatsApp <input name="phone" required></label>
      <label>Kode referensi <input name="ref_code" maxlength="8" placeholder="A1B2C3"></label>
      <label>Program <select name="program_slug"><option value="">-</option></select></label>
      <label class="full">Pesan / catatan <textarea name="message" rows="3"></textarea></label>
      <p class="error full" id="wa-error"></p>
      <div class="row full end">
        <button class="btn ghost" value="cancel" formnovalidate>Batal</button>
        <button class="btn primary" id="wa-submit" value="ok">Simpan</button>
      </div>
    </form>
  </dialog>
</body>
</html>
```

backend/app/static/dashboard/style.css — Tampilan dashboard (95 baris)

```
:root {
  --bg: #f4f6f8;
  --surface: #ffffff;
  --text: #1f2933;
  --muted: #64707d;
  --border: #e1e5ea;
  --primary: #0b7a4b;
  --primary-ink: #ffffff;
  --hot: #c2410c;
  --warm: #b7791f;
  --cold: #2b6cb0;
  --error: #c53030;
  --radius: 10px;
}
@media (prefers-color-scheme: dark) {
  :root {
    --bg: #12161b;
    --surface: #1b2128;
    --text: #e6eaee;
    --muted: #9aa5b1;
    --border: #2c343d;
    --primary: #2fae74;
    --primary-ink: #0d1a13;
    --hot: #f6875a;
    --warm: #e9b949;
    --cold: #6ba6f0;
    --error: #fc8181;
  }
}
* { box-sizing: border-box; }
```

```
[hidden] { display: none !important; }
body { margin: 0; background: var(--bg); color: var(--text); font: 14px/1.5 system-ui, -apple-system, "Segoe UI", sans-serif; }
h1, h2, h3 { margin: 0 0 12px; line-height: 1.25; }
h2 { font-size: 20px; } h3 { font-size: 15px; }
a { color: var(--primary); }
code { background: var(--bg); padding: 1px 4px; border-radius: 4px; }
.muted { color: var(--muted); }
.error { color: var(--error); min-height: 1em; }
.row { display: flex; gap: 8px; align-items: center; flex-wrap: wrap; }
.between { justify-content: space-between; margin-bottom: 12px; }
.end { justify-content: flex-end; }

.topbar { position: sticky; top: 0; z-index: 5; display: flex; gap: 16px; align-items: center; padding: 10px 16px; background: var(--surface); border-bottom: 1px solid var(--border); flex-wrap: wrap; }
.brand { color: var(--primary); }
#tabs { display: flex; gap: 4px; flex: 1; overflow-x: auto; }
#tabs a { padding: 6px 10px; border-radius: 6px; text-decoration: none; color: var(--muted); white-space: nowrap; }
#tabs a.active { background: var(--bg); color: var(--text); font-weight: 600; }
main { max-width: 1200px; margin: 0 auto; padding: 16px; }

.card { background: var(--surface); border: 1px solid var(--border); border-radius: var(--radius); padding: 16px; margin-bottom: 16px; overflow-x: auto; }
.tiles { display: grid; grid-template-columns: repeat(auto-fit, minmax(160px, 1fr)); gap: 12px; margin-bottom: 16px; }
.tile { background: var(--surface); border: 1px solid var(--border); border-radius: var(--radius); padding: 14px; }
.tile .v { font-size: 24px; font-weight: 700; }
.tile .l { color: var(--muted); font-size: 12px; }
.grid2 { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
@media (max-width: 800px) { .grid2 { grid-template-columns: 1fr; } }

.btn { border: 1px solid var(--border); background: var(--surface); color: var(--text); padding: 7px 12px; border-radius: 7px; cursor: pointer; font: inherit; text-decoration: none; display: inline-block; }
.btn.primary { background: var(--primary); border-color: var(--primary); color: var(--primary-ink); font-weight: 600; }
.btn.ghost { background: transparent; }
.btn.small { padding: 3px 8px; font-size: 12px; }
.btn:disabled { opacity: .6; cursor: default; }
input, select, textarea { font: inherit; color: var(--text); background: var(--surface); border: 1px solid var(--border); border-radius: 7px; padding: 7px 9px; width: 100%; }
.filters input, .filters select { width: auto; min-width: 150px; flex: 1; }
label { display: flex; flex-direction: column; gap: 4px; font-size: 13px; color: var(--muted); }
label.check { flex-direction: row; align-items: center; color: var(--text); }
label.check input { width: auto; }
.form-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 12px; align-items: end; }
.form-grid h3, .form-grid .full, .form-grid > p { grid-column: 1 / -1; }

.table { width: 100%; border-collapse: collapse; }
.table th, .table td { text-align: left; padding: 8px; border-bottom: 1px solid var(--border); vertical-align: top; }
.table th { color: var(--muted); font-weight: 600; font-size: 12px; }
.table tr.click { cursor: pointer; }
.table tr.click:hover { background: var(--bg); }
.num { text-align: right !important; font-variant-numeric: tabular-nums; }

.badge { display: inline-block; padding: 2px 8px; border-radius: 99px; font-size: 12px; font-weight: 600; border: 1px solid currentColor; }
.badge.hot { color: var(--hot); } .badge.warm { color: var(--warm); } .badge.cold { color: var(--cold); }
.bar { height: 6px; background: var(--border); border-radius: 3px; overflow: hidden; min-width: 60px; }
.bar > i { display: block; height: 100%; background: var(--primary); }
```

```

.login { min-height: 100vh; display: grid; place-items: center; padding: 16px; }
.login-card { width: 100%; max-width: 360px; display: flex; flex-direction: column; gap: 12px;
}

.dialog { border: 1px solid var(--border); border-radius: var(--radius); background: var(--sur
face); color: var(--text); padding: 20px; width: min(560px, calc(100vw - 32px)); }
.dialog.wide { width: min(900px, calc(100vw - 32px)); }
.dialog::backdrop { background: rgba(0, 0, 0, .45); }
.timeline { list-style: none; padding: 0; margin: 0; max-height: 320px; overflow-y: auto; }
.timeline li { padding: 6px 0; border-bottom: 1px dashed var(--border); }
.timeline .t { color: var(--muted); font-size: 12px; margin-right: 6px; }
.kv { display: grid; grid-template-columns: 160px 1fr; gap: 4px 12px; }
.kv dt { color: var(--muted); } .kv dd { margin: 0; }
.chips { display: flex; flex-wrap: wrap; gap: 6px; padding: 0; list-style: none; }
.chips li { background: var(--bg); border-radius: 99px; padding: 3px 10px; font-size: 12px; }
```

backend/app/static/dashboard/app.js — Logika dashboard admin (375 baris)

```
"use strict";

const STATUSES = ["baru", "dihubungi", "tertarik", "terdaftar", "bayar", "batal"];
const EVENT_TEXT = {
  page_view: "Membuka halaman", click_whatsapp: "Klik WhatsApp", click_cta: "Klik tombol",
  view_price: "Melihat biaya/jadwal", download_brochure: "Unduh brosur", form_start: "Mulai is
i form",
  form_submit: "Kirim form", site_search: "Mencari di situs",
};
const $ = (s, el = document) => el.querySelector(s);
const esc = (v) => String(v ?? "").replace(/[&<>"']/g, (c) => ({ "&": "&amp;", "<": "&lt;",
">": "&gt;", '"': "&quot;", "'": "&#39;" }[c]));
const pct = (v) => (v == null ? "-" : `${(v * 100).toFixed(1)}%`);
const fmtDate = (s) => (s ? new Date(s).toLocaleString("id-ID", { dateStyle: "medium", timeSty
le: "short" }) : "-");
const css = (name) => getComputedStyle(document.documentElement).getPropertyValue(name).trim
();

let token = null;
try { token = sessionStorage.getItem("ysii_token"); } catch (e) {}
let me = null, programs = [], charts = {}, leadPage = 1;

// ---------------- API ----------------
async function api(path, opts = {}) {
  const res = await fetch(path, {
    ...opts,
    headers: { "Content-Type": "application/json", ...(token ? { Authorization: `Bearer ${toke
n}` } : {}), ...(opts.headers || {}) },
  });
  if (res.status === 401) { logout(); throw new Error("Sesi berakhir"); }
  if (!res.ok) {
    let msg = res.statusText;
    try { const d = await res.json(); msg = typeof d.detail === "string" ? d.detail : JSON.str
ingify(d.detail); } catch (e) {}
    throw new Error(msg);
  }
  return res.status === 204 ? null : res.json();
}

// ---------------- auth ----------------
function logout() {
  token = null;
  try { sessionStorage.removeItem("ysii_token"); } catch (e) {}
  $("#app-view").hidden = true;
  $("#login-view").hidden = false;
}

$("#login-form").addEventListener("submit", async (e) => {
  e.preventDefault();
```

```
  const f = new FormData(e.target);
  $("#login-error").textContent = "";
  try {
    const d = await api("/v1/auth/login", { method: "POST", body: JSON.stringify(Object.fromEntries(f)) });
    token = d.access_token;
    try { sessionStorage.setItem("ysii_token", token); } catch (e) {}
    boot();
  } catch (err) { $("#login-error").textContent = err.message; }
});
$("#logout").addEventListener("click", logout);

// ---------------- routing ----------------
function route() {
  const page = (location.hash || "#ringkasan").slice(1);
  document.querySelectorAll("[data-page]").forEach((s) => (s.hidden = s.dataset.page !== page));
  document.querySelectorAll("#tabs a").forEach((a) => a.classList.toggle("active", a.getAttribute("href") === `#${page}`));
  ({ ringkasan: loadKpi, lead: loadLeads, program: loadPrograms, seo: loadSeo, model: loadModel }[page] || loadKpi)();
}
addEventListener("hashchange", route);

async function boot() {
  try { me = await api("/v1/admin/me"); } catch (e) { return logout(); }
  $("#login-view").hidden = true;
  $("#app-view").hidden = false;
  await refreshPrograms();
  route();
}

async function refreshPrograms() {
  programs = await api("/v1/admin/programs");
  const opts = programs.map((p) => `<option value="${esc(p.slug)}">${esc(p.name)}</option>`).join("");
  $('#lead-filter [name="program"]').innerHTML = `<option value="">Semua program</option>${opts}`;
  $('#wa-form [name="program_slug"]').innerHTML = `<option value="">-</option>${opts}`;
}

function chart(id, config) {
  if (!window.Chart) return;
  charts[id]?.destroy();
  Chart.defaults.color = css("--muted");
  Chart.defaults.borderColor = css("--border");
  charts[id] = new Chart(document.getElementById(id), config);
}

// ---------------- ringkasan ----------------
$("#kpi-days").addEventListener("change", loadKpi);
async function loadKpi() {
  const d = await api(`/v1/admin/kpi?days=${$("#kpi-days").value}`);
  const tiles = [
    ["Pengunjung", d.visitors.toLocaleString("id-ID")],
    ["Sesi organik (Google)", `${d.organic_sessions.toLocaleString("id-ID")} / ${d.sessions.toLocaleString("id-ID")}`],
    ["Lead baru", d.leads],
    ["Menjadi peserta", d.converted],
    ["Konversi pengunjung → lead", pct(d.visitor_to_lead_rate)],
    ["Konversi lead → peserta", pct(d.lead_to_participant_rate)],
    ["Lead HOT", d.segments.hot || 0],
  ];
  $("#kpi-tiles").innerHTML = tiles.map(([l, v]) => `<div class="tile"><div class="v">${esc(v)}</div><div class="l">${esc(l)}</div></div>`).join("");

  chart("chart-daily", {
    type: "line",
```

```
    data: {
      labels: d.series.map((s) => s.date.slice(5)),
      datasets: [
        { label: "Sesi", data: d.series.map((s) => s.sessions), borderColor: css("--cold"), tension: 0.3, yAxisID: "y" },
        { label: "Sesi organik", data: d.series.map((s) => s.organic), borderColor: css("--primary"), tension: 0.3, yAxisID: "y" },
        { label: "Lead", data: d.series.map((s) => s.leads), borderColor: css("--hot"), type: "bar", backgroundColor: css("--hot"), yAxisID: "y1" },
      ],
    },
    options: { interaction: { mode: "index", intersect: false }, scales: { y: { beginAtZero: true }, y1: { beginAtZero: true, position: "right", grid: { drawOnChartArea: false } } } },
  });
  const seg = ["hot", "warm", "cold"];
  chart("chart-segments", {
    type: "bar",
    data: { labels: ["Hot", "Warm", "Cold"], datasets: [{ label: "Lead", data: seg.map((s) => d.segments[s] || 0), backgroundColor: seg.map((s) => css(`--${s}`)) }] },
    options: { plugins: { legend: { display: false } }, scales: { y: { beginAtZero: true, ticks: { precision: 0 } } } },
  });
  $("#sources-table").innerHTML = "<tr><th>Sumber / medium</th><th class='num'>Sesi</th></tr>" +
    (d.sources.map(([s, n]) => `<tr><td>${esc(s)}</td><td class="num">${n}</td></tr>`).join("") || "<tr><td colspan=2 class='muted'>Belum ada data</td></tr>");
  const m = d.model;
  $("#model-summary").innerHTML = m.metrics
    ? `<dl class="kv"><dt>Algoritma</dt><dd>${esc(m.algorithm)}</dd><dt>Tugas</dt><dd>${esc(m.task_label)}</dd><dt>F1 (CV)</dt><dd>${m.metrics.f1}</dd><dt>AUC (CV)</dt><dd>${m.metrics.roc_auc ?? "-"}</dd><dt>Akurasi (CV)</dt><dd>${m.metrics.accuracy}</dd><dt>Dilatih</dt><dd>${fmtDate(m.trained_at)}</dd></dl>`
    : `<p>Memakai <b>skor berbasis aturan</b> karena data latih belum cukup. Model ML dapat dilatih di tab <a href="#model">Model ML</a>.</p>`;
}

// ---------------- lead ----------------
$('#lead-filter [name="status"]').innerHTML = `<option value="berjalan" selected>Masih berjalan</option><option value="">Semua status</option>${STATUSES.map((s) => `<option>${s}</option>`).join("")}`;
let filterTimer;
$("#lead-filter").addEventListener("input", () => { clearTimeout(filterTimer); filterTimer = setTimeout(() => { leadPage = 1; loadLeads(); }, 300); });
$("#lead-filter").addEventListener("submit", (e) => e.preventDefault());
$("#prev").addEventListener("click", () => { if (leadPage > 1) { leadPage--; loadLeads(); } });
$("#next").addEventListener("click", () => { leadPage++; loadLeads(); });

function waLink(phone, name) {
  if (!phone) return "";
  const text = encodeURIComponent(`Halo ${name || ""}, terima kasih sudah menghubungi Yayasan Sinergi Inspirasi Indonesia. `);
  return `<a class="btn small" target="_blank" rel="noopener" href="https://wa.me/${esc(phone)}?text=${text}">WhatsApp</a>`;
}
function scoreCell(l) {
  if (l.score == null) return "<span class='muted'>-</span>";
  return `<span class="badge ${esc(l.segment)}">${esc(l.segment)}</span> ${(l.score * 100).toFixed(0)}%<div class="bar"><i style="width:${(l.score * 100).toFixed(0)}%"></i></div>`;
}

async function loadLeads() {
  const params = new URLSearchParams(new FormData($("#lead-filter")));
  params.set("page", leadPage);
  params.set("limit", 25);
  const d = await api(`/v1/admin/leads?${params}`);
  const pages = Math.max(1, Math.ceil(d.total / d.limit));
  if (leadPage > pages) { leadPage = pages; return loadLeads(); }
```

```
  $("#lead-count").textContent = `${d.total} lead · halaman ${d.page} dari ${pages}`;
  $("#prev").disabled = leadPage <= 1;
  $("#next").disabled = leadPage >= pages;
  $("#lead-table").innerHTML =
    "<tr><th>Skor</th><th>Nama</th><th>Kontak</th><th>Program</th><th>Kanal</th><th>Masuk</th><th>Status</th><th></th></tr>" +
    (d.items.map((l) => `<tr class="click" data-id="${l.lead_id}">
      <td>${scoreCell(l)}</td><td>${esc(l.name || "-")}</td>
      <td>${esc(l.phone || "")}<br><span class="muted">${esc(l.email || "")}</span></td>
      <td>${esc(l.program_name || l.program_slug || "-")}</td><td>${esc(l.channel)}</td>
      <td>${fmtDate(l.created_at)}</td>
      <td><select data-status="${l.lead_id}" aria-label="Status">${STATUSES.map((s) => `<option ${s === l.status ? "selected" : ""}>${s}</option>`).join("")}</select></td>
      <td>${waLink(l.phone, l.name)}</td></tr>`).join("") || "<tr><td colspan=8 class='muted'>Belum ada lead</td></tr>");
}

$("#lead-table").addEventListener("change", async (e) => {
  const id = e.target.dataset.status;
  if (!id) return;
  try { await api(`/v1/admin/leads/${id}`, { method: "PATCH", body: JSON.stringify({ status: e.target.value }) }); }
  catch (err) { alert(err.message); }
});
$("#lead-table").addEventListener("click", (e) => {
  if (e.target.closest("select, a")) return;
  const tr = e.target.closest("tr[data-id]");
  if (tr) openLead(tr.dataset.id);
});

async function openLead(id) {
  const d = await api(`/v1/admin/leads/${id}`);
  const l = d.lead, v = d.visitor, s = d.score;
  const extra = l.extra ? Object.entries(l.extra).map(([k, val]) => `<dt>${esc(k)}</dt><dd>${esc(val)}</dd>`).join("") : "";
  $("#lead-detail").innerHTML = `
    <div class="row between"><h2>${esc(l.name || "Lead #" + l.lead_id)}</h2><button class="btn ghost" id="close-lead" type="button">Tutup</button></div>
    <div class="grid2">
      <div>
        <dl class="kv">
          <dt>Skor konversi</dt><dd>${scoreCell(l)}</dd>
          <dt>Kontak</dt><dd>${esc(l.phone || "-")} ${waLink(l.phone, l.name)}<br>${esc(l.email || "")}</dd>
          <dt>Program</dt><dd>${esc(l.program_name || l.program_slug || "-")}</dd>
          <dt>Kanal</dt><dd>${esc(l.channel)}</dd><dt>Masuk</dt><dd>${fmtDate(l.created_at)}</dd>
          <dt>Pesan</dt><dd>${esc(l.message || "-")}</dd>${extra}
          ${v ? `<dt>Sumber pertama</dt><dd>${esc(v.first_source)} / ${esc(v.first_medium)}</dd>
          <dt>Landing page</dt><dd>${esc(v.first_landing)}</dd><dt>Perangkat</dt><dd>${esc(v.device_type || "-")}</dd>
          <dt>Kode WA</dt><dd>${esc(v.ref_code)}</dd>` : "<dt>Riwayat</dt><dd class='muted'>Tidak terhubung dengan data kunjungan</dd>"}
        </dl>
        ${s ? `<h3 style="margin-top:12px">Faktor pendorong skor</h3><ul class="chips">${s.reasons.map((r) => `<li>${esc(r)}</li>`).join("") || "<li>-</li>"}</ul><p class="muted">Model: ${esc(s.model_version)}</p>` : ""}
        <form id="lead-edit" class="form-grid" style="margin-top:12px">
          <label>Status <select name="status">${STATUSES.map((x) => `<option ${x === l.status ? "selected" : ""}>${x}</option>`).join("")}</select></label>
          <label class="full">Catatan follow-up <textarea name="notes" rows="3">${esc(l.notes || "")}</textarea></label>
          <div class="row full end"><button class="btn ghost" type="button" id="rescore">Hitung ulang skor</button><button class="btn primary" type="submit">Simpan</button></div>
        </form>
      </div>
      <div>
```

```
        <h3>Riwayat kunjungan (${d.sessions.length} sesi)</h3>
        <table class="table"><tr><th>Waktu</th><th>Sumber</th><th class="num">Hal.</th><th cla
ss="num">Detik</th><th class="num">Scroll</th></tr>
          ${d.sessions.map((x) => `<tr><td>${fmtDate(x.started_at)}</td><td>${esc(x.source)}/
${esc(x.medium)}${x.utm_term ? `<br><span class="muted">${esc(x.utm_term)}</span>` : ""}</td><
td class="num">${x.page_count}</td><td class="num">${x.active_seconds}</td><td class="num">
${x.max_scroll_pct}%</td></tr>`).join("")}
        </table>
        <h3>Aktivitas</h3>
        <ul class="timeline">${d.timeline.map((t) => `<li><span class="t">${fmtDate(t.ts)}</sp
an>${esc(EVENT_TEXT[t.type] || t.type)} <span class="muted">${esc(t.label || t.page || "")}</s
pan></li>`).join("") || "<li class='muted'>-</li>"}</ul>
      </div>
    </div>`;
  const dlg = $("#lead-dialog");
  $("#close-lead").onclick = () => dlg.close();
  $("#rescore").onclick = async () => { await api(`/v1/admin/leads/${id}/rescore`, { method:
"POST" }); openLead(id); loadLeads(); };
  $("#lead-edit").onsubmit = async (e) => {
    e.preventDefault();
    await api(`/v1/admin/leads/${id}`, { method: "PATCH", body: JSON.stringify(Object.fromEntr
ies(new FormData(e.target))) });
    dlg.close();
    loadLeads();
  };
  if (!dlg.open) dlg.showModal();
}

$("#export-csv").addEventListener("click", async (e) => {
  e.preventDefault();
  const res = await fetch("/v1/admin/export/leads.csv", { headers: { Authorization: `Bearer
${token}` } });
  if (!res.ok) return alert("Gagal mengunduh");
  const a = document.createElement("a");
  a.href = URL.createObjectURL(await res.blob());
  a.download = `lead-ysii-${new Date().toISOString().slice(0, 10)}.csv`;
  a.click();
  URL.revokeObjectURL(a.href);
});

$("#btn-add-wa").addEventListener("click", () => { $("#wa-form").reset(); $("#wa-error").textC
ontent = ""; $("#wa-dialog").showModal(); });
$("#wa-submit").addEventListener("click", async (e) => {
  e.preventDefault();
  const form = $("#wa-form");
  if (!form.reportValidity()) return;
  const body = Object.fromEntries(new FormData(form));
  Object.keys(body).forEach((k) => body[k] === "" && delete body[k]);
  try {
    await api("/v1/admin/leads", { method: "POST", body: JSON.stringify(body) });
    $("#wa-dialog").close();
    loadLeads();
  } catch (err) { $("#wa-error").textContent = err.message; }
});

// ---------------- program ----------------
async function loadPrograms() {
  await refreshPrograms();
  $("#program-table").innerHTML = "<tr><th>Slug</th><th>Nama</th><th>Kategori</th><th class='n
um'>Harga</th><th>Batch</th><th>Aktif</th><th></th></tr>" +
    (programs.map((p) => `<tr><td><code>${esc(p.slug)}</code></td><td>${p.url ? `<a href="${es
c(p.url)}" target="_blank" rel="noopener">${esc(p.name)}</a>` : esc(p.name)}</td>
      <td>${esc(p.category || "-")}</td><td class="num">${p.price != null ? Number(p.price).to
LocaleString("id-ID") : "-"}</td>
      <td>${esc(p.next_batch || "-")}</td><td>${p.active ? "Ya" : "Tidak"}</td>
      <td class="row"><button class="btn small" data-edit="${esc(p.slug)}" type="button">Ubah
</button><button class="btn small" data-del="${esc(p.slug)}" type="button">Hapus</button></td>
</tr>`).join("") ||
```

```
      "<tr><td colspan=7 class='muted'>Belum ada program. Tambahkan di bawah.</td></tr>");
}
$("#program-table").addEventListener("click", async (e) => {
  const edit = e.target.dataset.edit, del = e.target.dataset.del;
  if (edit) {
    const p = programs.find((x) => x.slug === edit);
    const f = $("#program-form");
    ["slug", "name", "category", "price", "url", "next_batch"].forEach((k) => (f.elements[k].value = p[k] ?? ""));
    f.elements.active.checked = p.active;
    f.scrollIntoView({ behavior: "smooth" });
  }
  if (del && confirm(`Hapus program ${del}?`)) { await api(`/v1/admin/programs/${del}`, { method: "DELETE" }); loadPrograms(); }
});
$("#program-form").addEventListener("submit", async (e) => {
  e.preventDefault();
  const f = e.target;
  const body = {
    slug: f.elements.slug.value.trim(), name: f.elements.name.value.trim(), category: f.elements.category.value || null,
    price: f.elements.price.value ? Number(f.elements.price.value) : null, url: f.elements.url.value || null,
    next_batch: f.elements.next_batch.value || null, active: f.elements.active.checked,
  };
  try { await api(`/v1/admin/programs/${body.slug}`, { method: "PUT", body: JSON.stringify(body) }); f.reset(); loadPrograms(); }
  catch (err) { alert(err.message); }
});

// ---------------- SEO ----------------
async function loadSeo() {
  const d = await api("/v1/admin/seo?days=28");
  if (!d.available) {
    $("#seo-body").innerHTML = `<div class="card"><p>Belum ada data Google Search Console. Atur <code>GSC_CREDENTIALS_FILE</code> lalu jalankan
      <code>python -m jobs.gsc_etl --days 90</code> (lihat README).</p></div>`;
    return;
  }
  const rows = (list, key) => list.map((r) => `<tr><td>${esc(r[key])}</td><td class="num">${r.clicks}</td><td class="num">${r.impressions}</td><td class="num">${pct(r.ctr)}</td><td class="num">${r.position ?? "-"}</td></tr>`).join("");
  const head = (k) => `<tr><th>${k}</th><th class="num">Klik</th><th class="num">Impresi</th><th class="num">CTR</th><th class="num">Posisi</th></tr>`;
  $("#seo-body").innerHTML = `
    <div class="tiles">
      <div class="tile"><div class="v">${d.clicks.toLocaleString("id-ID")}</div><div class="l">Klik organik (28 hari)</div></div>
      <div class="tile"><div class="v">${d.impressions.toLocaleString("id-ID")}</div><div class="l">Impresi</div></div>
      <div class="tile"><div class="v">${pct(d.ctr)}</div><div class="l">CTR</div></div>
      <div class="tile"><div class="v">${d.avg_position ?? "-"}</div><div class="l">Rata-rata posisi</div></div>
    </div>
    <div class="card"><h3>Tren harian</h3><canvas id="chart-seo" height="90"></canvas></div>
    <div class="grid2">
      <div class="card"><h3>Kata kunci teratas</h3><table class="table">${head("Kata kunci")}${rows(d.top_queries, "query")}</table></div>
      <div class="card"><h3>Halaman teratas</h3><table class="table">${head("Halaman")}${rows(d.top_pages, "page")}</table></div>
    </div>
    ${d.low_ctr_pages.length ? `<div class="card"><h3>Perlu perbaikan judul/meta (impresi tinggi, CTR &lt; 2%)</h3><table class="table">${head("Halaman")}${rows(d.low_ctr_pages, "page")}</table></div>` : ""}`;
  chart("chart-seo", {
    type: "line",
    data: { labels: d.series.map((s) => s.date.slice(5)), datasets: [
```

```
      { label: "Klik", data: d.series.map((s) => s.clicks), borderColor: css("--primary"), yAxisID: "y", tension: 0.3 },
      { label: "Impresi", data: d.series.map((s) => s.impressions), borderColor: css("--cold"), yAxisID: "y1", tension: 0.3 },
    ] },
    options: { interaction: { mode: "index", intersect: false }, scales: { y: { beginAtZero: true }, y1: { beginAtZero: true, position: "right", grid: { drawOnChartArea: false } } } },
  });
}

// ---------------- model ----------------
let modelPoll;
const ALGO_NAME = { logistic_regression: "Logistic Regression (baseline)", random_forest: "Random Forest", gradient_boosting: "Gradient Boosting" };

function comparisonCard(m) {
  const r = m.report;
  const best = r.comparison[r.best_algorithm];
  return `<div class="card"><h3>${esc(r.task_label)} — model aktif <code>${esc(m.version)}</code></h3>
    <p class="muted">${r.n_samples} data (${r.n_positive} positif) · ${r.cv_folds}-fold stratified CV · data uji 20% · dilatih ${fmtDate(m.trained_at)}</p>
    <table class="table"><tr><th>Algoritma</th><th class="num">Akurasi</th><th class="num">Presisi</th><th class="num">Recall</th><th class="num">F1</th><th class="num">AUC</th><th class="num">F1 (uji)</th><th class="num">AUC (uji)</th></tr>
    ${Object.entries(r.comparison).map(([k, v]) => `<tr${k === r.best_algorithm ? ' style="font-weight:700"' : ""}><td>${esc(ALGO_NAME[k] || k)}${k === r.best_algorithm ? " ★" : ""}</td>
      <td class="num">${v.cv.accuracy}</td><td class="num">${v.cv.precision}</td><td class="num">${v.cv.recall}</td><td class="num">${v.cv.f1} ± ${v.cv.f1_std}</td><td class="num">${v.cv.roc_auc ?? "-"}</td>
      <td class="num">${v.holdout.f1}</td><td class="num">${v.holdout.roc_auc ?? "-"}</td></tr>`).join("")}</table>
    <p class="muted">Confusion matrix model terbaik pada data uji [[TN, FP], [FN, TP]]: ${esc(JSON.stringify(best.holdout.confusion_matrix))}</p>
    <h3>Kepentingan fitur (permutation importance)</h3>
    <table class="table"><tr><th>Fitur</th><th class="num">Penurunan AUC</th></tr>${r.feature_importance.slice(0, 10).map((f) => `<tr><td>${esc(f.feature)}</td><td class="num">${f.importance}</td></tr>`).join("")}</table>
    <button class="btn small" data-rules="${esc(m.task)}" type="button">Kembali ke skor aturan untuk tugas ini</button></div>`;
}

async function loadModel() {
  const d = await api("/v1/admin/model");
  const t = d.training;
  let status = "";
  if (t.running) status = "<p><b>Pelatihan sedang berjalan…</b></p>";
  else if (t.last_error) status = `<p class="error">${esc(t.last_error)}</p>`;
  else if (t.last_result) status = `<p>Hasil pelatihan terakhir:</p><ul>${Object.entries(t.last_result).map(([k, v]) => `<li><b>${esc(d.tasks[k])}</b>: ${esc(v)}</li>`).join("")}</ul>`;

  const activeCards = Object.keys(d.tasks).map((task) => {
    const m = d.models.find((x) => x.task === task && x.is_active);
    return m ? comparisonCard(m)
      : `<div class="card"><h3>${esc(d.tasks[task])}</h3><p>Memakai <b>skor berbasis aturan</b> (${esc(d.rule_version)}). Model ML aktif setelah ada minimal ${d.min_samples} contoh positif dan ${d.min_samples} negatif.</p></div>`;
  }).join("");

  $("#model-body").innerHTML = `
    <div class="card">${status}
      <p>Data berlabel: <b>${d.labeled.leads}</b> lead (${d.labeled.linked} terhubung dengan data kunjungan), <b>${d.labeled.converted}</b> menjadi peserta, <b>${d.labeled.batal}</b> batal.
      Status lead yang diisi tim di tab Lead menjadi data latih baru setiap kali model dilatih ulang (otomatis setiap Senin 02.00 WIB).</p>
    </div>
    ${activeCards}
```

```
    <div class="card"><h3>Riwayat model</h3><table class="table"><tr><th>Versi</th><th>Tugas</th><th>Dilatih</th><th class="num">Data</th><th class="num">Positif</th><th class="num">F1 CV</th><th class="num">AUC CV</th><th></th></tr>
      ${d.models.map((m) => { const cv = m.report.comparison[m.algorithm].cv; return `<tr><td><code>${esc(m.version)}</code></td><td>${esc(m.task)}</td><td>${fmtDate(m.trained_at)}</td><td class="num">${m.n_samples}</td><td class="num">${m.n_positive}</td>
        <td class="num">${cv.f1}</td><td class="num">${cv.roc_auc ?? "-"}</td><td>${m.is_active ? "<b>aktif</b>" : `<button class="btn small" data-activate="${esc(m.version)}" type="button">Aktifkan</button>`}</td></tr>`; }).join("") || "<tr><td colspan=8 class='muted'>Belum ada model</td></tr>"}
    </table></div>`;
  clearTimeout(modelPoll);
  if (t.running) modelPoll = setTimeout(loadModel, 3000);
}
$("#model-body").addEventListener("click", async (e) => {
  const v = e.target.dataset.activate, rules = e.target.dataset.rules;
  if (v) { await api(`/v1/admin/model/${encodeURIComponent(v)}/activate`, { method: "POST" }); loadModel(); }
  if (rules) { await api(`/v1/admin/model/deactivate?task=${encodeURIComponent(rules)}`, { method: "POST" }); loadModel(); }
});
$("#btn-retrain").addEventListener("click", async () => {
  try {
    await api("/v1/admin/model/retrain", { method: "POST", body: JSON.stringify({ task: $("#retrain-task").value, force: $("#retrain-force").checked }) });
    setTimeout(loadModel, 500);
  } catch (err) { alert(err.message); }
});

// ---------------- start ----------------
if (token) boot(); else logout();
```

integration/wordpress/ysii-tracker/ysii-tracker.php — Plugin WordPress (113 baris)

```
<?php
/**
 * Plugin Name: YSII Tracker (SEO & ML)
 * Description: Menghubungkan website dengan Sistem Promosi Digital Terintegrasi SEO & Machine Learning: tracking perilaku (dengan persetujuan), kode referensi WhatsApp, salinan form pendaftaran, dan widget rekomendasi [ysii_reco].
 * Version: 1.0.0
 * Author: Tim PTI Politeknik Negeri Medan
 * License: GPLv2 or later
 */

if (!defined('ABSPATH')) {
    exit;
}

const YSII_OPTION = 'ysii_tracker_settings';

function ysii_settings() {
    return wp_parse_args(get_option(YSII_OPTION, []), [
        'api_url'      => 'https://api.ysii.org',
        'privacy_url'  => home_url('/kebijakan-privasi/'),
        'consent'      => 'required',
        'mirror_forms' => 'auto',
        'whatsapp'     => '',
        'daftar_url'   => home_url('/pendaftaran/'),
    ]);
}

// 1) Pasang tracker.js di <head>
add_action('wp_head', function () {
    if (is_admin() || is_user_logged_in() && current_user_can('edit_posts')) {
        return; // jangan rekam admin/editor yang sedang mengelola situs
    }
    $s = ysii_settings();
```

```
    printf(
        '<script src="%s" defer data-privacy-url="%s" data-consent="%s" data-mirror-forms="%
s"></script>' . "\n",
        esc_url(rtrim($s['api_url'], '/') . '/static/tracker.js'),
        esc_url($s['privacy_url']),
        esc_attr($s['consent']),
        esc_attr($s['mirror_forms'])
    );
    // Tandai halaman program agar slug-nya terbaca (meta "ysii:program" diisi dari custom fie
ld)
    if (is_singular()) {
        $slug = get_post_meta(get_the_ID(), 'ysii_program', true);
        if ($slug) {
            printf('<meta name="ysii:program" content="%s">' . "\n", esc_attr($slug));
        }
    }
}, 1);

// 2) Shortcode widget rekomendasi: [ysii_reco]
add_shortcode('ysii_reco', function () {
    $s = ysii_settings();
    return sprintf(
        '<div id="ysii-reco" data-whatsapp="%s" data-daftar-url="%s"></div>',
        esc_attr($s['whatsapp']),
        esc_url($s['daftar_url'])
    );
});

// 3) Halaman pengaturan: Pengaturan > YSII Tracker
add_action('admin_menu', function () {
    add_options_page('YSII Tracker', 'YSII Tracker', 'manage_options', 'ysii-tracker', 'ysii_s
ettings_page');
});

add_action('admin_init', function () {
    register_setting('ysii_tracker', YSII_OPTION, [
        'sanitize_callback' => function ($in) {
            return [
                'api_url'      => esc_url_raw($in['api_url'] ?? ''),
                'privacy_url'  => esc_url_raw($in['privacy_url'] ?? ''),
                'consent'      => in_array($in['consent'] ?? '', ['required', 'implied'], tru
e) ? $in['consent'] : 'required',
                'mirror_forms' => in_array($in['mirror_forms'] ?? '', ['auto', 'marked', 'of
f'], true) ? $in['mirror_forms'] : 'auto',
                'whatsapp'     => preg_replace('/\D/', '', $in['whatsapp'] ?? ''),
                'daftar_url'   => esc_url_raw($in['daftar_url'] ?? ''),
            ];
        },
    ]);
});

function ysii_settings_page() {
    $s = ysii_settings();
    ?>
    <div class="wrap">
        <h1>YSII Tracker</h1>
        <form method="post" action="options.php">
            <?php settings_fields('ysii_tracker'); ?>
            <table class="form-table" role="presentation">
                <tr><th><label for="api_url">URL API</label></th>
                    <td><input class="regular-text" id="api_url" name="<?php echo YSII_OPTION;
?>[api_url]" value="<?php echo esc_attr($s['api_url']); ?>"></td></tr>
                <tr><th><label for="privacy_url">Halaman kebijakan privasi</label></th>
                    <td><input class="regular-text" id="privacy_url" name="<?php echo YSII_OPT
ION; ?>[privacy_url]" value="<?php echo esc_attr($s['privacy_url']); ?>"></td></tr>
                <tr><th>Persetujuan cookie</th>
                    <td><select name="<?php echo YSII_OPTION; ?>[consent]">
```

```
                        <option value="required" <?php selected($s['consent'], 'required'); ?>
>Tampilkan banner (disarankan, UU PDP)</option>
                        <option value="implied" <?php selected($s['consent'], 'implied'); ?>>T
anpa banner</option>
                    </select></td></tr>
                <tr><th>Salin form ke sistem lead</th>
                    <td><select name="<?php echo YSII_OPTION; ?>[mirror_forms]">
                        <option value="auto" <?php selected($s['mirror_forms'], 'auto'); ?>>Ot
omatis (form yang punya kolom HP/email)</option>
                        <option value="marked" <?php selected($s['mirror_forms'], 'marked'); ?
>Hanya form bertanda data-ysii-form</option>
                        <option value="off" <?php selected($s['mirror_forms'], 'off'); ?>>Mati
</option>
                    </select></td></tr>
                <tr><th><label for="whatsapp">No. WhatsApp admin</label></th>
                    <td><input id="whatsapp" name="<?php echo YSII_OPTION; ?>[whatsapp]" value
="<?php echo esc_attr($s['whatsapp']); ?>" placeholder="6281234567890"></td></tr>
                <tr><th><label for="daftar_url">URL halaman pendaftaran</label></th>
                    <td><input class="regular-text" id="daftar_url" name="<?php echo YSII_OPTI
ON; ?>[daftar_url]" value="<?php echo esc_attr($s['daftar_url']); ?>"></td></tr>
            </table>
            <?php submit_button(); ?>
        </form>
        <p>Halaman program: isi <em>custom field</em> <code>ysii_program</code> dengan slug pr
ogram (sama dengan di dashboard),
        atau gunakan URL berpola <code>/program/&lt;slug&gt;</code>. Tambahkan shortcode <code
>[ysii_reco]</code> untuk widget rekomendasi.</p>
    </div>
    <?php
}
```

backend/Dockerfile — Image kontainer aplikasi (15 baris)

```
FROM python:3.13-slim

ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
RUN useradd --create-home appuser && mkdir -p /app/models && chown -R appuser /app
USER appuser

EXPOSE 8000
# 1 worker: status pelatihan model & rate limit disimpan di memori proses
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--proxy-headers", "--f
orwarded-allow-ips", "*"]
```

docker-compose.yml — Orkestrasi layanan (69 baris)

```
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: ysii
      POSTGRES_USER: ysii
      POSTGRES_PASSWORD: ${DB_PASSWORD:?isi DB_PASSWORD di .env}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ysii -d ysii"]
      interval: 10s
      retries: 5

  api:
```

```
    build: ./backend
    restart: unless-stopped
    env_file: .env
    environment:
      DATABASE_URL: postgresql+psycopg://ysii:${DB_PASSWORD}@db:5432/ysii
      MODEL_DIR: /app/models
    volumes:
      - models:/app/models
      - ./secrets:/secrets:ro
    depends_on:
      db:
        condition: service_healthy

  scheduler:
    build: ./backend
    restart: unless-stopped
    command: python -m jobs.scheduler
    env_file: .env
    environment:
      DATABASE_URL: postgresql+psycopg://ysii:${DB_PASSWORD}@db:5432/ysii
      MODEL_DIR: /app/models
    volumes:
      - models:/app/models
      - ./secrets:/secrets:ro
    depends_on:
      db:
        condition: service_healthy

  nginx:
    image: nginx:1.27-alpine
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    environment:
      API_DOMAIN: ${API_DOMAIN:-api.ysii.org}
    volumes:
      - ./nginx/templates:/etc/nginx/templates:ro
      - ./nginx/certbot/conf:/etc/letsencrypt:ro
      - ./nginx/certbot/www:/var/www/certbot:ro
    depends_on:
      - api

  certbot:
    image: certbot/certbot:v2.11.0
    profiles: ["certbot"]
    volumes:
      - ./nginx/certbot/conf:/etc/letsencrypt
      - ./nginx/certbot/www:/var/www/certbot

volumes:
  pgdata:
  models:
```

nginx/templates/default.conf.template — Konfigurasi server web (24 baris)

```
# Template diproses oleh image nginx (envsubst): ${API_DOMAIN} diganti dari environment.

# Resolver DNS Docker: alamat container API di-resolve ulang setiap request,
# sehingga nginx tidak 502 setelah container API dibuat ulang (update/deploy).
resolver 127.0.0.11 valid=10s ipv6=off;
map $host $api_upstream { default api:8000; }

server {
    listen 80;
    server_name ${API_DOMAIN};

    location /.well-known/acme-challenge/ { root /var/www/certbot; }
```

```
13
14      # Sebelum sertifikat ada, layani HTTP saja (lihat README langkah HTTPS)
15      location / {
16          if (-f /etc/letsencrypt/live/${API_DOMAIN}/fullchain.pem) { return 301 https://$host$r
    equest_uri; }
17          proxy_pass http://$api_upstream;
18          proxy_set_header Host $host;
19          proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
20          proxy_set_header X-Forwarded-Proto $scheme;
21      }
22  }
23
24  include /etc/nginx/conf.d/https.inc*;
```